

메타코딩의 자율주행 개발 일기

"AI를 처음 만나는 사람도, 끝까지 읽으면 이해하게 되는 이야기"

한 줄의 코드도 모르던 메타코딩이 5주 만에 자율주행 AI를 만들기까지, 실패와 성장의 여정을 따라가다 보면 어느새 강화학습의 핵심이 손에 잡힙니다.

목차

- 시작하기 전에: 실습 환경 준비
- 프롤로그: 운명을 바꾼 월요일 아침
- 0화: if문의 벽 (v0)
- 1화: AI에게 눈을 달아주다 (v1)
- 2화: 넘어져야 걷는 법을 안다 (v2)
- 3화: 가짜 90점의 진실 (v3)
- 4화: 처음엔 크게, 나중엔 섬세하게 (v4)
- 5화: 현장에서 살아남기 (v5)
- 番外편: 망치 너머의 도구들 (v6)
- 에필로그: 실패는 다음 버전의 씨앗이다

시작하기 전에: 실습 환경 준비

이 책은 **이론서이자 실습서**입니다! 각 화마다 직접 시뮬레이터를 실행해보며 AI가 학습하는 과정을 눈으로 확인할 수 있습니다. 아래 순서대로 환경을 준비해주세요.

1단계: 파이썬(Python) 설치

AI 시뮬레이터는 파이썬으로 만들어져 있습니다. 파이썬이 설치되어 있지 않다면 아래 링크에서 다운로드해주세요.

- **다운로드:** <https://www.python.org/downloads/>
- **버전:** Python 3.8 이상 권장
- 설치 시 **"Add Python to PATH"** 체크박스를 반드시 체크해주세요!

설치 확인:

```
python --version
# Python 3.x.x 가 나오면 성공!
```

2단계: 커서(Cursor) 설치 + 파이썬 익스텐션

코드를 편하게 보고 실행하려면 **커서(Cursor)** 에디터를 추천합니다.

- **다운로드:** <https://cursor.sh/>
- 설치 후 **Extensions(확장)** 탭에서 **"Python"** 을 검색하여 설치해주세요.

- Python 익스텐션이 있어야 코드 자동완성, 실행 등이 편리합니다.

3단계: GitHub에서 프로젝트 다운로드

이 책의 모든 실습 코드는 아래 GitHub 저장소에 있습니다.

```
git clone https://github.com/metacoding-ai-project/self-driving-car-v1.git
```

또는 GitHub 페이지에서 "**Code** → **Download ZIP**" 버튼을 눌러 다운로드한 후 압축을 풀어주세요.

- **GitHub 주소:** <https://github.com/metacoding-ai-project/self-driving-car-v1>

4단계: 프로젝트 열고 실행하기

1. 커서(Cursor)에서 **File** → **Open Folder** 로 다운로드한 **self-driving-car-v1** 폴더를 엽니다.
2. 왼쪽 탐색기에서 **simulator-v0**, **simulator-v1**, ... **simulator-v6** 폴더를 확인합니다.
3. 터미널을 열고 (**Ctrl+`** 또는 **Cmd+`**) 아래처럼 실행합니다:

```
# 예: v0 실행하기
cd simulator-v0
python main.py
```

💡 필요한 패키지가 없다는 에러가 나면:

```
pip install pygame torch numpy
```

📁 프로젝트 폴더 구조

```
self-driving-car-v1/
├── simulator-v0/    ← 0화: 규칙 기반 (if문)
├── simulator-v1/    ← 1화: 첫 번째 DQN
├── simulator-v2/    ← 2화: 다양한 맵 학습
├── simulator-v3/    ← 3화: 훈련셋/테스트셋 분리
├── simulator-v4/    ← 4화: Learning Rate 스케줄링
├── simulator-v5/    ← 5화: 캐싱 시스템
└── simulator-v6/    ← 번외편: 의사결정 트리 / 랜덤 포레스트
```

각 폴더에는 두 가지 핵심 파일이 있습니다:

- **train.py** — AI를 학습시키는 파일 (v1~v5)
- **main.py** — 학습된 AI를 실행하는 파일 (v0~v5)
- **config.py** — 시뮬레이터 속도 등 설정 파일

💡 필요한 추가 패키지 (번외편용):

```
pip install scikit-learn matplotlib
```

⚙️ 속도 조절 팁 (모든 버전 공통)

각 버전의 `config.py` 파일에서:

```
SHOW_TRAINING = False # 화면 없이 빠르게 학습 (권장)
```

이렇게 설정하면 **화면 표시 없이 CPU 최대 속도**로 학습합니다! 학습 과정을 눈으로 보고 싶을 때만 `True`로 바꿔주세요.

또한 모든 버전의 `config.py`에서 시뮬레이터 속도를 바꿀 수 있습니다:

```
CURRENT_SPEED = SPEED_NORMAL # 아래 값 중 선택!

SPEED_SLOW = 10 # 천천히 관찰 (초보자용)
SPEED_NORMAL = 30 # 보통 속도 (추천)
SPEED_FAST = 60 # 빠르게 (학습 이해했을 때)
SPEED_ULTRA = 120 # 초고속 (테스트용)
```

💡 **준비 완료!** 이제 각 화를 읽으면서 직접 실행해보세요. 시뮬레이터를 돌려보면 이해가 훨씬 빨라집니다!

프롤로그: 운명을 바꾼 월요일 아침

📅 어느 월요일 아침

사무실에 들어서자마자 팀장님이 손짓했습니다.

"메타코딩 씨, 잠깐 와봐요."

회의실 문을 열었더니, 테이블 위에 작은 로봇 청소기가 놓여 있었습니다. 바퀴가 달린 작은 플라스틱 덩어리. 웬지 그게 오늘부터 메타코딩의 운명을 바꿔놓을 것 같은 불길한 예감이 들었습니다.

"이거 보이죠? 우리 회사에서 자율주행 청소 로봇을 만들기로 했어요."

팀장님이 로봇 청소기를 톡톡 두드렸습니다.

"메타코딩 씨가 AI 파트를 담당해주세요."

순간 메타코딩의 머릿속이 하얘졌습니다.

'AI? 나를요?'

강화학습은 대학교 때 교수님이 PPT로 설명해주신 적이 있었습니다. 솔직히 절반도 이해를 못 했습니다. '벨만 방정식'이라는 글자가 화면에 떴을 때 바로 졸음이 밀려왔던 기억만 생생합니다. 그걸로 실제 제품을 만든다고

요?

"걱정 마세요. 처음부터 실제 로봇에 넣을 필요 없어요. 일단 컴퓨터 시뮬레이터로 먼저 만들어봐요. 2주 안에 간단한 데모만 보여주시면 됩니다!"

팀장님은 밝게 웃으며 회의실을 나갔습니다. 메타코딩은 로봇 청소기와 둘이 남았습니다. 로봇 청소기의 동그란 버튼이 마치 눈처럼 보였습니다. '너도 나처럼 길을 모르는구나.'

🏠 집에 돌아와서

그날 밤, 메타코딩은 노트북 앞에 앉아 인터넷을 뒤지기 시작했습니다.

"자율주행 강화학습 입문"

검색 결과에 **DQN**이라는 단어가 계속 나왔습니다. Deep Q-Network. 풀어보면 "깊은 Q 네트워크"인데, 이름만으로는 도무지 감이 잡히지 않았습니다.

그런데 읽다 보니 흥미로운 점이 있었습니다. 2013년에 DeepMind라는 회사가 이 DQN으로 아타리 게임을 사람보다 잘 하는 AI를 만들었다는 것입니다. 게임 AI랑 청소 로봇이 비슷하지 않을까? 둘 다 어떤 공간에서 목표를 향해 이동하는 거잖아.

위키피디아, 블로그, 논문... 여기저기 읽어보니 한 가지가 명확해졌습니다.

DQN = 강화학습 + 딥러닝

"강화학습? 딥러닝? 그게 대체 뭐지?"

시계를 보니 밤 11시. 하지만 메타코딩의 눈은 오히려 더 반짝였습니다. 모르는 게 나오면 잠이 안 오는 체질이니까.

"좋아, 일단 큰 그림부터 보자."

🗺️ AI 세계의 전체 지도

메타코딩은 "AI 학습 종류"를 검색했습니다. 그리고 드디어 전체 그림이 보이기 시작했습니다.

AI가 배우는 방식은 딱 **3가지**입니다. 비유하면 이렇습니다.

1. 지도학습 (Supervised Learning) — "선생님이 답을 알려주는 공부"

시험 문제집에 정답지가 있다고 생각해 보세요. 문제를 풀고, 정답을 확인하고, 틀린 걸 고칩니다. 이걸 수천 번 반복하면? 새로운 문제도 잘 풀게 됩니다.

쉽게 말하면, "**이건 고양이야**" "**이건 강아지야**" 하고 정답을 계속 알려주면, AI가 나중에 처음 보는 사진도 구별하게 되는 겁니다.

2. 비지도학습 (Unsupervised Learning) — "정답 없이 스스로 분류하는 공부"

이번엔 정답지가 없습니다. 대신 수백 장의 동물 사진을 AI한테 던져주면, AI가 스스로 "음, 이 그룹은 비슷하게 생겼고, 저 그룹은 또 다르네?" 하고 나눕니다. 정답을 안 알려줘도 패턴을 찾아내는 겁니다.

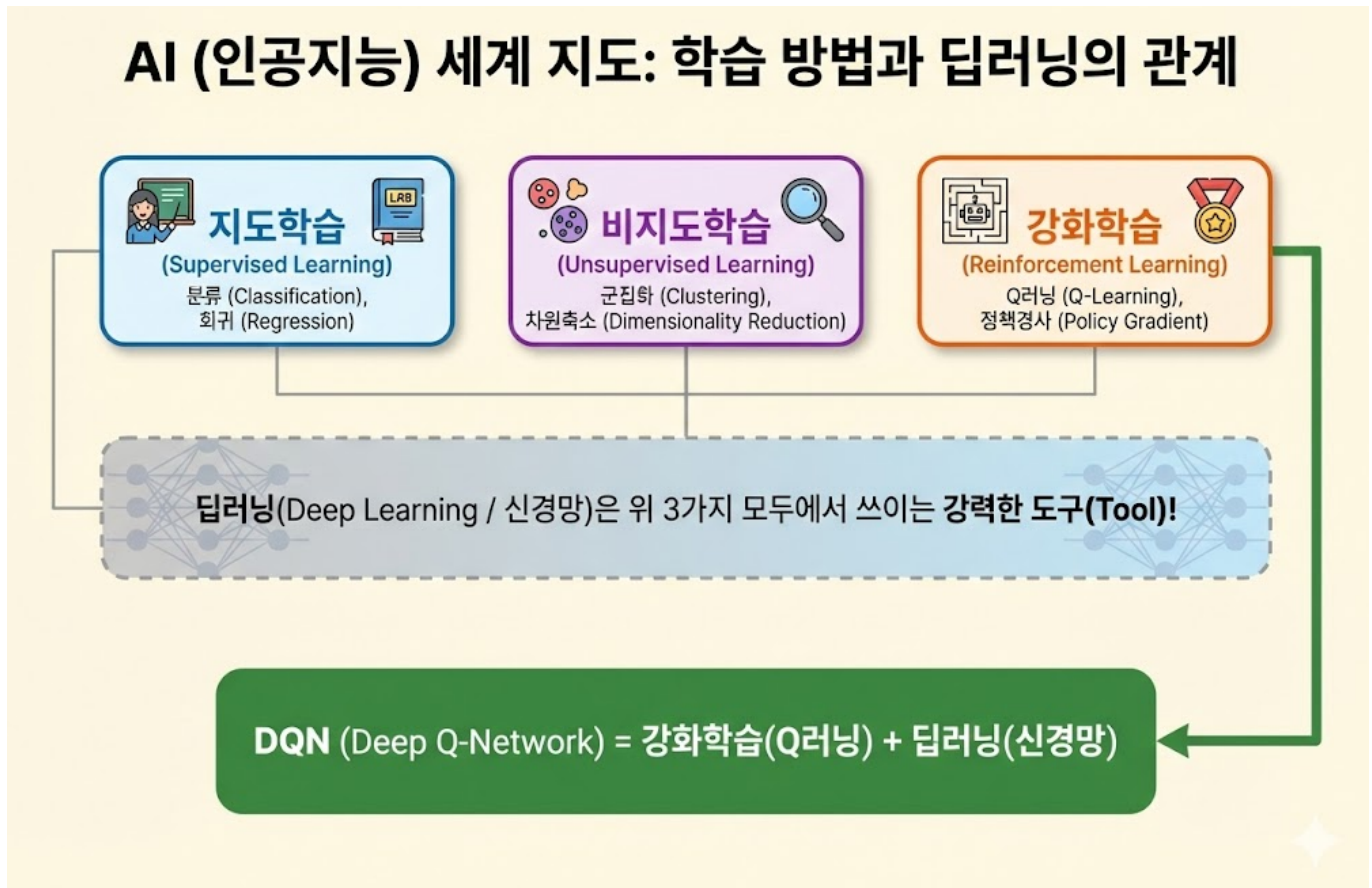
비유하면, 처음 가는 학교에서 아무 설명 없이 학생들을 관찰하다가 "아, 재네키리 친한 그룹이구나" 하고 파악하는 것과 같습니다.

3. 강화학습 (Reinforcement Learning) — "시행착오로 배우는 공부"

정답지도 없고, 사진도 없습니다. 대신 **직접 해보고**, 잘하면 **보상**을 받고, 못하면 **벌**을 받습니다. 이걸 수만 번 반복하면서 점점 잘하게 됩니다.

비유하면? 자전거 배우는 것과 똑같습니다. 아무도 "페달을 13.7도 각도로 밟아"라고 안 가르쳐줍니다. 그냥 타보고, 넘어지고, 또 타보고, 균형 잡히면 "오 됐다!" 하고... 그러다 어느 날 자연스럽게 타게 되는 겁니다.

메타코딩은 노트에 정리했습니다.



핵심 정리: 딥러닝은 AI의 "종류"가 아닙니다! 딥러닝은 **도구**입니다. 망치가 집도 짓고 가구도 만드는 것처럼, 딥러닝(신경망)은 지도학습에도, 비지도학습에도, 강화학습에도 쓸 수 있습니다. 우리 책에서는 **강화학습 + 딥러닝 = DQN**을 배웁니다!

"아하!" 메타코딩이 무릎을 쳅습니다. "딥러닝이 따로 있는 게 아니라, 강화학습에 딥러닝을 갖다 붙인 게 DQN 이구나!"

그런데 검색하다 보니 자꾸 **회귀**니 **분류**니 하는 단어가 나왔습니다. 이것도 알아야 할 것 같았습니다.

🌀 AI가 푸는 문제의 종류

메타코딩은 계속 읽었습니다. AI가 푸는 문제는 크게 보면 이렇게 나뉩니다.

1. 회귀 (Regression) — "숫자를 맞혀라!"

한 마디로, 숫자 하나를 **예측**하는 겁니다.

- "내일 기온은 몇 도?"

- "이 집 가격은 얼마?"
- "이 학생의 다음 시험 점수는 몇 점?"

자율주행으로 치면? "목적지까지 남은 거리는 몇 칸?" 같은 문제입니다.

2. 분류 (Classification) — "어느 쪽이야?"

한 마디로, **카테고리를 고르는** 겁니다.

- "이 메일은 스팸이야, 아니야?"
- "이 사진은 고양이야, 강아지야?"

자율주행으로 치면? "이 칸은 벽이야, 길이야?" 같은 문제입니다.

3. 강화학습 — "직접 해보고 배워라!"

회귀나 분류와는 좀 다릅니다. 정답 데이터가 없습니다. 대신 **환경 안에서 직접 행동**하고, 그 결과로 **보상이나 벌**을 받으면서 배웁니다. 바로 **이 책 전체**가 강화학습입니다!

한 마디 정리:

- 회귀 = "**숫자** 맞추기" (얼마? 몇 도? 몇 점?)
- 분류 = "**종류** 맞추기" (A야? B야?)
- 강화학습 = "**직접 해보며** 배우기" (보상과 벌!)

"그런데..." 메타코딩은 고개를 갸우뚱했습니다. "회귀랑 분류는 대체 어떻게 작동하는 거지?"

선형 회귀 — 직선 하나로 미래를 예측한다

회귀 중에서 가장 기본은 **선형 회귀**입니다. 쉽게 말하면 *****점들 사이에 직선 하나 긋기*****입니다.

메타코딩이 매일 걷는 걸음 수와 소모한 에너지를 기록했다고 생각해봅시다.

걸음 수 (x):	10	20	30	40	50
소모 에너지(y):	15	28	42	55	70

이걸 그래프에 점으로 찍으면, 거의 일직선!

→ 이 점들 사이에 가장 잘 맞는 직선을 긋는 게 선형 회귀!

직선 공식: $y = 1.4x + 1$

질문! 100걸음 걸으면 에너지가 얼마나 소모될까?

→ $y = 1.4 \times 100 + 1 = 141$ 에너지!

→ 아직 100걸음을 안 걸어봤는데 미래를 예측한 것!

한 마디로: 선형 회귀 = 직선 하나로 미래의 숫자를 예측하는 것! 중학교 때 배운 $y = ax + b$ 가 바로 이것입니다.

로지스틱 회귀 — 확률로 분류한다

분류 중에서 가장 기본은 **로지스틱 회귀**입니다. 핵심은 간단합니다. "**확률을 계산해서, 높으면 O, 낮으면 X**"

가장 쉬운 예시로 설명해봅시다. "**이 사람이 암에 걸릴 확률은?**"

원인(입력값):

- a. 흡연 여부 → 1 (흡연)
- b. 운동 빈도 → 0.2 (거의 안 함)
- c. 가족력 → 1 (있음)

이 a, b, c 값을 넣으면 → $y(\text{암 걸릴 확률}) = 87\%$

그런데 이 확률은 어떻게 만드는 걸까요?

$a \times \text{가중치} + b \times \text{가중치} + c \times \text{가중치} = \text{큰 숫자가 나옴}$

예: $1 \times 2.0 + 0.2 \times 0.5 + 1 \times 1.5 = 3.6$

문제: 3.6이라는 숫자가 나왔는데... 이게 확률인가요?

확률은 0~1(0%~100%) 사이여야 하잖아요!

여기서 **시그모이드(Sigmoid) 함수**가 등장합니다!

시그모이드가 하는 일:

어떤 숫자든 0과 1 사이의 확률로 바꿔준다!

3.6 → 시그모이드 통과 → 0.97 (97%) → "위험!"으로 분류

-2.0 → 시그모이드 통과 → 0.12 (12%) → "안전!"으로 분류

0.0 → 시그모이드 통과 → 0.50 (50%) → "반반!"

이 S자 모양의 곡선 = 시그모이드(Sigmoid) 함수!

다른 예시도 봅시다.

스팸 메일 판별:

- a. "무료" 단어 포함 여부 → 1
- b. 발신자가 연락처에 있나 → 0
- c. 링크 개수 → 5개

→ 시그모이드 통과 → 0.92 (92%) → "스팸!"으로 분류

한 마디로: 로지스틱 회귀 = 여러 원인(a, b, c)을 넣으면, 시그모이드 함수가 0~1 사이의 확률(y)로 바꿔 줘서 O/X를 판단하는 것! 이 시그모이드 함수, 기억해두세요. 1화에서 신경망을 배울 때 다시 등장합니다!

메타코딩은 모니터에서 눈을 떼고 기지개를 켜습니다. 시계를 보니 새벽 1시.

우리 프로젝트에 필요한 것:

- ☒ 강화학습 → 차가 직접 부딪혀보며 배운다
- ☒ 딥러닝 → 신경망으로 똑똑하게 판단한다
- ☒ DQN → 강화학습 + 딥러닝 = 우리가 만들 것!

참고로 알아둘 것:

- 🔗 회귀 → 숫자 예측 (직선 긋기)
- 🔗 분류 → 0/x 판단 (시그모이드)
- 🔗 시그모이드 → 1화에서 다시 만남!

"좋아. AI가 뭔지, 강화학습이 뭔지, 대충 감은 잡았어. 이제 진짜로 만들어보자!"

메타코딩은 노트에 첫 번째 줄을 적었습니다.

"Step 1: AI 없이 규칙만으로 만들어보자."

🌀 시뮬레이터 설계

다음날 출근해서 메타코딩은 시뮬레이터부터 설계했습니다. 진짜 로봇을 만들기 전에, 컴퓨터 화면에서 먼저 실험해보는 것이죠.

목표는 단순합니다:

- 시작점 ●(노란색)에서 출발
- 목적지 ■(초록색)까지 도달
- 벽 ■(빨간색)에 부딪히면 안 됨

자율주행 시뮬레이터 화면 구조



30×30 칸의 격자 세상. 이 작은 세상에서 자동차가 벽을 피해 목적지를 찾아야 합니다.

그런데 자동차를 어떻게 움직이게 할지가 문제였습니다. 메타코딩은 두 가지 방법을 생각했습니다.

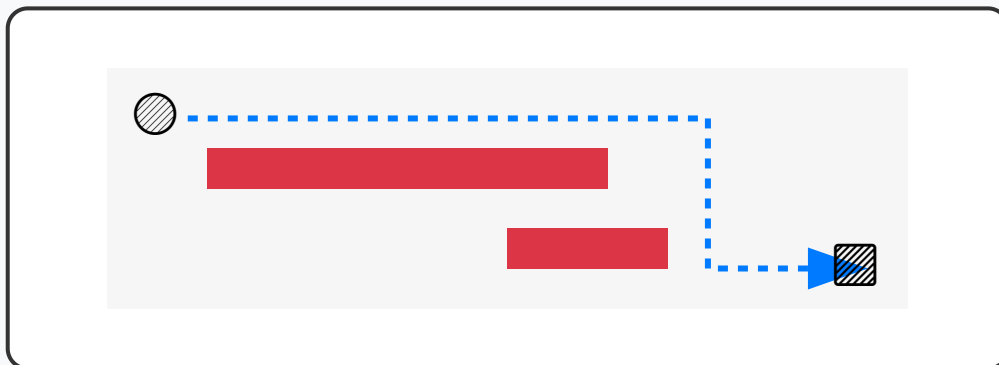
방법 1: 규칙		방법 2: AI
원리	"목적지가 오른쪽이면 오른쪽으로!"	"좋은 행동엔 칭찬, 나쁜 행동엔 꾸중"
장점	간단하고 빠르게 만들 수 있음	복잡한 상황도 스스로 해결
단점	예외 상황에 약함	만들기 어렵고 학습이 필요

"일단 방법 1부터 해보자. 그게 제일 쉬우니까."

0화: if문의 벽 (v0)

♀ 지금 배우는 것 → 규칙 기반 (AI 이전 단계)
지도학습 | 비지도학습 | 강화학습 | 딥러닝
← 여기서 시작!

🔗 0화: 규칙 기반 (if문) 주행 전략



전략: "목적지 방향으로 if 문 이동!"

단점: 복잡한 미로에서는 뱅뱅 돌며 갇힘 (국소 최적화)

💡 가장 단순한 아이디어

메타코딩은 종이에 규칙을 적었습니다.

규칙 1: 목적지가 오른쪽에 있으면 → 오른쪽으로 **규칙 2:** 목적지가 아래쪽에 있으면 → 아래쪽으로 **규칙 3:** 벽에 막혔으면 → 다른 방향 시도

마치 나침반을 손에 들고 항상 목적지 방향으로 걸어가되, 벽이 나오면 돌아가는 것이죠. 너무 간단해 보였지만, 메타코딩은 일단 코드를 짭니다.

🔑 v0 코드 소개

```
def decide_action(self, environment):
    goal_x, goal_y = environment.goal_pos
```

```

# 목적지까지의 거리 계산
dx = goal_x - self.x    # 양수 = 오른쪽, 음수 = 왼쪽
dy = goal_y - self.y    # 양수 = 아래쪽, 음수 = 위쪽

# 우선순위 방향 결정
preferred = []
if dx > 0: preferred.append(RIGHT)
elif dx < 0: preferred.append(LEFT)
if dy > 0: preferred.append(DOWN)
elif dy < 0: preferred.append(UP)

# 우선순위로 벽이 없는 방향으로 이동
for direction in preferred:
    next_x, next_y = 다음_위치(direction)
    if not environment.is_wall(next_x, next_y):
        return direction    # 이 방향으로!

```

코드는 짧고 명쾌했습니다. "목적지가 어디 있나 보고, 그 방향으로 가되, 벽이면 다른 데로 가라." 사람이 보기에도 자연스러운 논리죠.

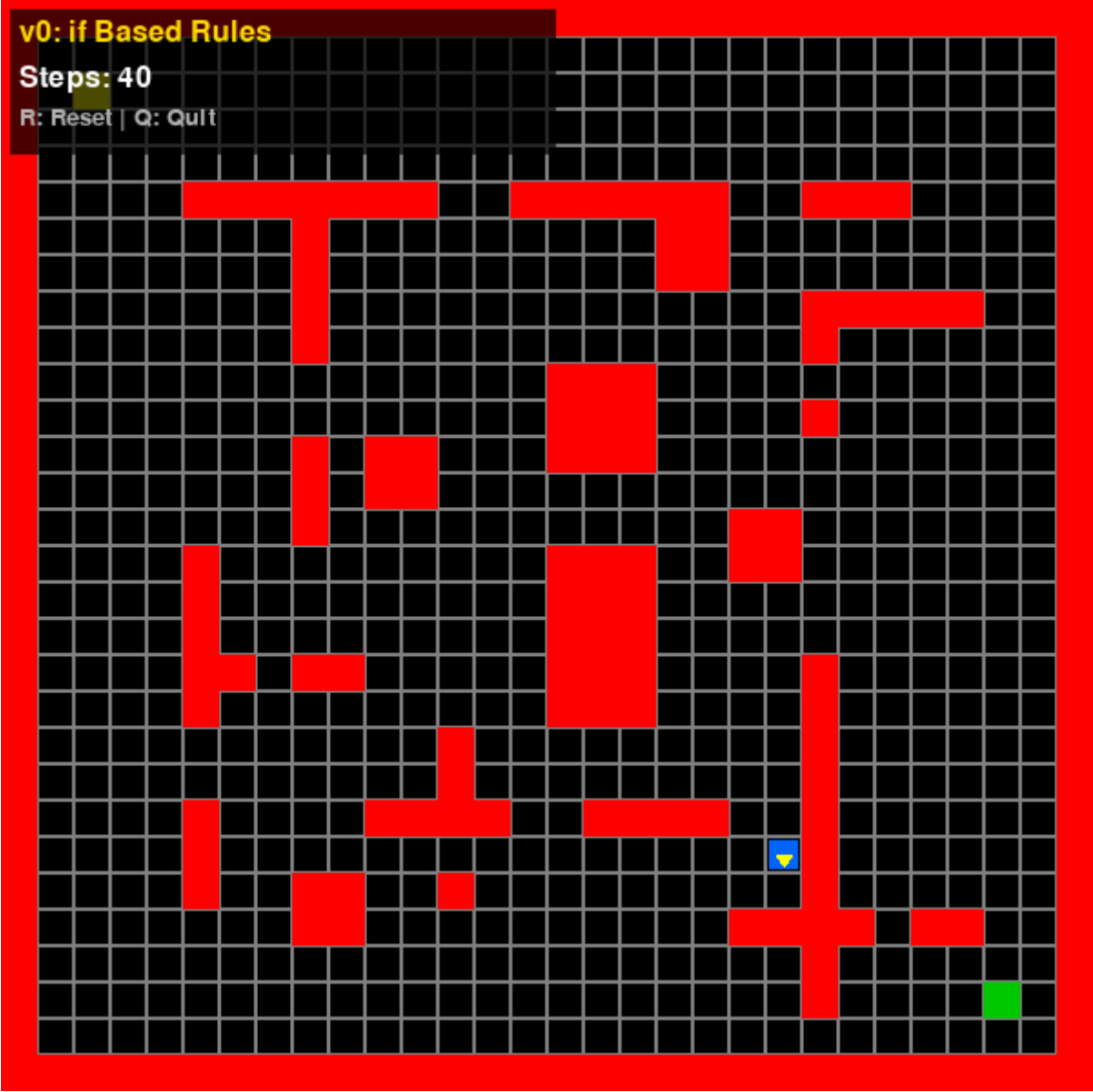
 직접 실행해보기!

 지금 바로 실행해볼 수 있습니다!

```

cd simulator-v0
python main.py

```



🎮 v0 실행해보니...

단순한 맵에서는 놀랍도록 잘 작동했습니다.

[단순한 맵 - v0 성공!]

A diagram of a simple 1D environment. It consists of a horizontal track with a car icon at the left end. A series of right-pointing arrows indicates the car's path. At the end of the track, a downward arrow points to a shaded square representing the goal. The track is bounded by vertical walls on the left and right sides.

결과: ☒ 성공! (87걸음)

"오! 되는데?"

메타코딩은 신이 났습니다. 화면 속 파란 자동차가 목적지까지 매끄럽게 달려갔으니까요.

- 1000번 실패해도 1001번째도 똑같이 실패!
- 경험으로부터 배우지 않음!

☑ AI (강화학습):

- "이 상황에서 오른쪽으로 갔더니 벽이었어."
- "다음엔 위로 먼저 가봐야겠어."
- 실패할수록 더 잘해짐! 점점 발전!

규칙 기반 시스템에는 **"기억"이 없습니다.** 어제의 실패를 오늘의 교훈으로 바꿀 수 없습니다. 하지만 AI는 다릅니다. AI는 실패를 기억하고, 그 기억에서 패턴을 찾아내고, 다음에는 더 나은 선택을 합니다.

☁ 메타코딩의 깨달음

메타코딩은 노트에 적었습니다.

"규칙만으로는 한계가 있다. 복잡한 환경에서는 스스로 배우는 AI가 필요하다."

사람도 마찬가지입니다. 처음 운전을 배울 때 "빨간불이면 멈춰라"는 규칙만으로는 복잡한 도로를 달릴 수 없습니다. 수백 번의 경험을 쌓으면서 "이런 상황에서는 이렇게"라는 감각이 생기는 것처럼, AI도 경험이 필요합니다.

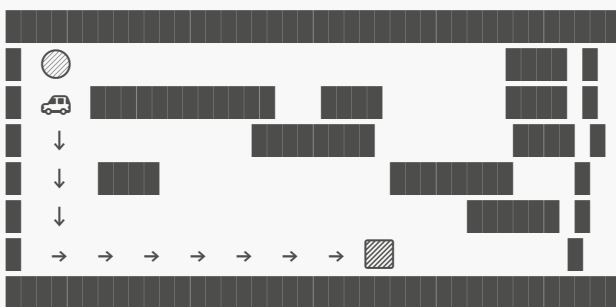
"좋아, 이제 진짜 AI를 만들어보자!"

하지만 AI를 만들기 전에, 먼저 결정해야 할 것이 있었습니다. **AI에게 뭘 보여줄 것인가?** 사람도 눈이 있어야 운전할 수 있듯이, AI에게도 "눈"이 필요합니다. 어떤 정보를 얼마만큼 보여줄지부터 정해야 합니다.

1화: AI에게 눈을 달아주다 (v1)

- 📍 지금 배우는 것 → 강화학습 + 신경망(딥러닝)
- 지도학습 | 비지도학습 | 강화학습 ★ | 딥러닝 ★

🔗 1화에서 만드는 것: simulator-v1



특징: DQN으로 처음 학습! (500 에피소드)
문제: 항상 같은 경로만 (과적합!)
성공률: 80% (같은 맵에서만!)

🤖 AI에게 뭘 보여줘야 하지?

본격적으로 AI를 만들기 전에, 메타코딩은 한 가지 중요한 질문과 마주했습니다.

🤖 "AI에게 뭘 알려줘야 하지?"

"전체 맵 정보를 다 줘야 할까?"

만약 전체 맵을 통째로 AI에게 보여준다면 어떻게 될까요? AI는 그 맵을 사진 찍듯 외워버립니다. "왼쪽에서 세 번째 칸에서 오른쪽으로 꺾어"라고 외우는 거죠. 당연히 맵이 바뀌면 아무것도 못 합니다.

마치 서울에서 부산까지 가는 길을 "신호등 237개째에서 좌회전"이라고 외우는 것과 같습니다. 도로 공사라도 하면 바로 길을 잃어버리죠.

"나침반처럼 방향만 알려주면 어떨까? '지금 목적지가 저쪽이야. 그리고 바로 주변에 벽이 있어.' 이 정도면 충분하지 않을까?"

이 아이디어가 핵심이었습니다.

📊 AI가 받는 정보: 상태 벡터 (State Vector)

상태 벡터란 현재 상황을 숫자들의 묶음으로 표현한 것입니다. AI는 글자나 그림이 아니라, **오직 숫자만** 이해합니다. 그래서 "지금 앞에 벽이 있어"를 "1"로, "앞에 벽이 없어"를 "0"으로 변환해서 알려줍니다.

우리 AI는 총 **11개의 숫자**를 받습니다:

AI가 보는 세상: 상태 벡터 (State Vector)

①=0	②=1 (벽)	③=0
④=0		⑤=0
⑥=1 (벽)	⑦=0	⑧=0

⑨ 현재 방향: 오른쪽(→)

⑩ 목적지 X 방향: +0.6

⑪ 목적지 Y 방향: +0.4

[0, 1, 0, 0, 0, 1, 0, 0, 1, +0.6, +0.4]

총 11개의 숫자로 이루어진 리스트(벡터)

왜 하필 11개일까요?

번호	정보	값 예시	역할
①~⑧	주변 8칸 벽 유무	0 또는 1	"지금 주변이 어떤가?"
⑨	현재 바라보는 방향	0~3 (0=위, 1=오른쪽, 2=아래, 3=왼쪽)	"나는 어디를 보고 있나?"
⑩	목적지 X 방향 거리	-1.0 ~ +1.0	"목적지가 왼쪽? 오른쪽?"
⑪	목적지 Y 방향 거리	-1.0 ~ +1.0	"목적지가 위? 아래?"

이 11개 숫자가 AI의 "눈" 입니다. 전체 지도가 아니라, "지금 내 주변"과 "목적지 방향"만 아는 것이죠.

💡 **쉽게 말하면:** AI는 "바로 옆에 벽이 있나?" + "목적지가 어느 쪽이야?" 이 두 가지만 알고 운전합니다. 마치 안대를 한 채로 손과 나침반만으로 길을 찾는 것과 비슷해요!

📦 전처리(Preprocessing): AI의 통역사

방금 한 일을 AI 용어로 말하면 **전처리(Data Preprocessing)** 라고 합니다!

AI는 글자나 그림이 이해하지 못합니다. **오직 숫자만** 이해합니다. 그래서 세상의 모든 것을 숫자로 변환해줘야 합니다. 이 변환 작업이 바로 전처리입니다.

전처리(Preprocessing) = 세상을 AI가 이해할 수 있는 숫자로 변환!

전처리 전: "앞에 벽이 있고, 목적지는 오른쪽 아래야"

전처리 후: [0, 1, 0, 0, 0, 1, 0, 0, 1, +0.6, +0.4]

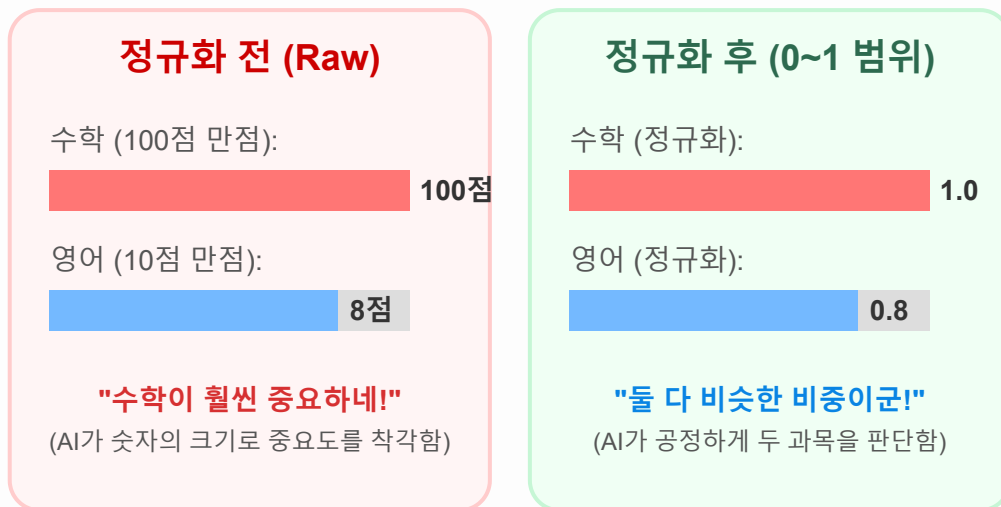
→ AI는 숫자만 이해하니까, 모든 것을 숫자로 바꿔주는 것이 전처리!

📏 정규화(Normalization): 숫자의 스케일 맞추기

전처리에서 한 가지 더 중요한 것이 있습니다. **숫자의 크기를 비슷하게 맞추는 것**입니다. 이것을 **정규화(Normalization)** 라고 합니다.

왜 필요할까요? 이렇게 생각해 보세요. 시험을 두 과목 봤는데, 수학은 **100점 만점**이고 영어는 **10점 만점**이에요.

정규화(Normalization): 숫자의 만점 통일



※ 숫자 크기가 다르면 AI가 착각합니다!

AI도 똑같은 착각을 합니다! 숫자가 큰 쪽이 더 중요하다고 오해해요.

✗ 정규화 안 한 경우:

벽 유무: 0 또는 1 (작은 숫자)
 거리: 450 (큰 숫자!)
 → AI: "450이 1보다 크니까 거리만 신경 쓸래!"
 → 벽이 바로 앞에 있는데 무시! ☆ 충돌!

☑ 정규화 한 경우:

벽 유무: 0 또는 1 (0~1 범위)
 거리: 0.6 (0~1 범위로 변환!)
 → AI: "둘 다 비슷한 크기네, 둘 다 잘 살펴보자!"
 → 벽도 보고, 거리도 보고, 공정하게 판단! ☑

우리 시뮬레이터에서도 목적지 방향을 -1.0 ~ +1.0 범위로 변환했죠? 그게 바로 정규화입니다!

💡 한 마디로:

- **전처리** = 세상을 숫자로 변환 (AI의 통역사)
- **정규화** = 숫자의 크기를 비슷하게 맞추기 (만점을 통일!)

🎁 보상 시스템: AI에게 칭찬과 꾸중하기

정보만 준다고 AI가 배우는 건 아닙니다. **"잘했어!"(보상)** 와 **"안 돼!"(패널티)** 가 있어야 무엇이 좋고 나쁜지 알 수 있습니다.

강아지를 훈련시킬 때를 생각해 보세요. "앉아!"를 했을 때 간식을 주면, 강아지는 "앉으면 좋은 일이 생기는구나!"를 배우니다. 쓰레기통을 뒤지면 혼나니까, "쓰레기통은 건드리면 안 되는구나!"를 배우죠.

AI도 마찬가지로:

```
if 목적지 도달:
    reward = +100 # "완벽해! 바로 이거야!"
elif 목적지에 가까워짐:
    reward = +0.5 # "좋아, 방향은 맞아!"
elif 벽에 충돌:
    reward = -10 # "앗! 벽이야, 조심해!"
elif 시간 초과:
    reward = -50 # "너무 오래 걸렸어!"
```

이걸 **보상 함수(Reward Function)** 라고 합니다. AI의 선생님 같은 존재입니다.

여기서 중요한 점이 있습니다. 보상을 어떻게 설계하느냐에 따라 AI의 행동이 완전히 달라집니다. 예를 들어 "벽에 부딪히면 -10"이 아니라 "-1000"으로 했다면? AI는 벽이 너무 무서워서 아예 움직이지 않으려 할 수도 있습니다. 반대로 "-1"로 했다면? "벽에 좀 부딪혀도 괜찮지"라며 벽에 마구 부딪힐 수 있습니다.

좋은 보상 함수를 설계하는 것은 AI에서 매우 중요한 기술입니다. 마치 좋은 선생님이 적절한 칭찬과 꾸중의 균형을 잡는 것처럼요.

📁 드디어 AI를 만들다!

메타코딩은 팔을 걷어붙이고 코드를 짜기 시작했습니다. 검색을 거듭할수록 **DQN(Deep Q-Network)** 이 가장 적합하다는 확신이 들었습니다.

"DQN... 이름은 어렵지만, 핵심은 간단해."

한 줄로 요약하면 이렇습니다:

"AI에게 '이 상황에서 이 행동을 하면 얼마나 좋을까?'를 계산하는 뇌를 만들어주는 것."

여기서 **"얼마나 좋을까?"**를 숫자로 나타낸 것이 바로 **Q값(Q-value)** 입니다.

💡 Q값이란? — "이 행동의 예상 점수"

Q값은 마치 **레스토랑 리뷰 점수** 같은 것입니다.

맛집 앱을 보고 저녁 메뉴를 고르는 상황:

치킨집 → 리뷰 점수: 4.8 ★★★★★
피자집 → 리뷰 점수: 3.2 ★★★
한식집 → 리뷰 점수: 2.1 ★★

→ "치킨집이 4.8로 가장 높으니까 치킨 먹으러 가자!"

우리 AI도 똑같습니다:

직진 → Q값: 8.5 ★★★★★ "직진하면 좋은 일이 생길 거야!"
우회전 → Q값: 3.2 ★★★ "우회전은 별로야"

좌회전 → Q값: 2.1 ★★ "좌회전은 위험해"

→ "직진이 8.5로 가장 높으니까 직진!"

처음에는 이 Q값이 엉터리입니다. 아직 아무것도 모르니까요. 하지만 수백 번 직접 해보면서 "이 상황에서 직진 하니까 벽이었어 → Q값을 낮추자", "이 상황에서 우회전하니까 목적지에 가까워졌어 → Q값을 올리자"를 반복 하면, Q값이 점점 정확한 리뷰 점수가 됩니다.

Q값 = "이 상황에서 이 행동을 하면 미래에 얼마나 좋은 결과가 나올지" 예측한 점수!

Q값이 뭔지 알았으니, 이제 이 Q값을 어떻게 계산하는지 볼 차례입니다. 그 계산기가 바로 **신경망**입니다!

🧠 **신경망(Neural Network): AI의 뇌**

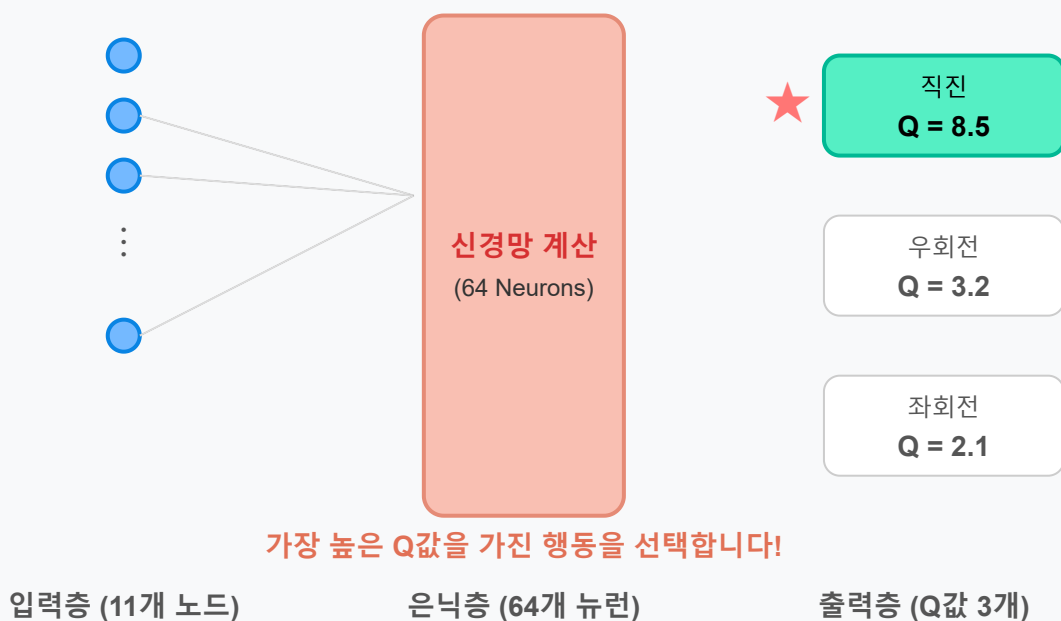
한 마디로: 숫자를 넣으면, 계산을 거쳐서, 답이 나오는 기계! 입력(주변 상황 11개) → 계산(뉴런 64개) → 출력(직진/우회전/좌회전)

사람의 뇌에는 약 860억 개의 **뉴런(신경세포)** 이 있습니다. 이 뉴런들이 서로 신호를 주고받으면서 "생각"을 합니다. 신경망은 이것 흉내 낸 거예요. 물론 860억 개는 무리고, 우리 AI는 소박하게 **78개의 뉴런**으로 시작합니다.

메타코딩은 이렇게 이해했습니다:

"자판기 같은 거구나! 동전(입력)을 넣으면 내부에서 뭔가 굴러가고, 음료(출력)가 나오는 거잖아. 신경망도 숫자를 넣으면 내부에서 계산하고, '직진해!' 같은 답이 나오는 거네!"

DQN 신경망 구조와 Q값



11개 숫자를 넣으면, 자동으로 "직진/우회전/좌회전" 중 어느 것이 가장 좋은지 **Q값**으로 알려줍니다.

🔗 **뉴런 하나는 어떻게 계산할까?**

은닉층의 뉴런 하나가 하는 일을 들여다봅시다. 생각보다 단순합니다.

먼저 입력값 11개가 각각 무슨 의미인지 다시 봅시다:

번호	정보	값 예시	역할
①~⑧	주변 8칸 벽 유무	0 또는 1	"지금 주변이 어떤가?"
⑨	현재 바라보는 방향	0~3 (0=위, 1=오른쪽, 2=아래, 3=왼쪽)	"나는 어디를 보고 있나?"
⑩	목적지 X 방향 거리	-1.0 ~ +1.0	"목적지가 왼쪽? 오른쪽?"
⑪	목적지 Y 방향 거리	-1.0 ~ +1.0	"목적지가 위? 아래?"

이제 이 상태값으로 뉴런이 어떻게 계산하는지 봅시다!

[뉴런 하나의 계산 과정 - 3단계]

1단계: 각 입력(상태값)에 "가중치(weight)"를 곱한다

상태값(의미)	입력값	× 가중치(w)	= 결과
① 위쪽 벽 유무	0 (없음)	× 0.3	= 0
② 오른쪽위 벽 유무	1 (있음!)	× 0.8	= 0.8
③ 오른쪽 벽 유무	0 (없음)	× 0.1	= 0
④ 오른쪽아래 벽 유무	0 (없음)	× 0.5	= 0
⑤ 아래쪽 벽 유무	0 (없음)	× 0.2	= 0
⑥ 왼쪽아래 벽 유무	1 (있음!)	× 0.7	= 0.7
⑦ 왼쪽 벽 유무	0 (없음)	× 0.4	= 0
⑧ 왼쪽위 벽 유무	0 (없음)	× 0.1	= 0
⑨ 바라보는 방향	1 (오른쪽)	× 0.6	= 0.6
⑩ 목적지 X방향 거리	+0.6	× 0.9	= 0.54
⑪ 목적지 Y방향 거리	+0.4	× 0.3	= 0.12

※ 가중치(0.3, 0.8, 0.1...)는 처음에는 완전 랜덤!
 학습을 수천 번 반복하면서 점점 좋은 값으로 바뀝니다.

2단계: 전부 더하고, 편향(bias)도 더한다

$$\begin{aligned}
 \text{합계} &= 0 + 0.8 + 0 + 0 + 0 + 0.7 + 0 + 0 \\
 &\quad + 0.6 + 0.54 + 0.12 + \text{편향}(0.1) \\
 &= 2.86
 \end{aligned}$$

"잠깐, 왜 전부 더해? 오른쪽위에 벽 있고, 왼쪽아래에 벽 있으면 그걸 보고 피해야 하는 거 아냐? 왜 더하기를 해?"

이게 가장 헷갈리는 부분입니다! 핵심을 말씀드리면:

뉴런 하나는 "어디로 가라"를 결정하는 게 아닙니다! 뉴런 하나는 "지금 상황이 어떤 패턴인지 감지"하는 센서입니다!

예시의 뉴런이 하는 일:

- ② 오른쪽위 벽 → 0.8 (높은 가중치! 이 정보를 중요하게 봄)
- ⑥ 왼쪽아래 벽 → 0.7 (높은 가중치! 이 정보도 중요하게 봄)
- ⑩ 목적지 x거리 → 0.54

합계 = 2.86 ← 이걸 "답"이 아님!
 ← "위험한 벽이 대각선으로 있는 상황이다!"를 감지한 것!

이 뉴런은 "대각선에 벽이 있는 패턴"을 감지하는 센서가 된 것!
 벽이 없는 상황이면? → 합계가 낮게 나옴 → "이 패턴 아님"
 벽이 대각선에 있으면? → 합계가 높게 나옴 → "이 패턴 맞음!"

그러면 실제로 "어디로 가라"는 누가 결정하나요?

이것이 핵심입니다!

뉴런 1개가 결정하는 게 아니라, 64개 뉴런이 팀으로 일합니다!

뉴런 1번: "대각선에 벽이 있다!" → 2.86
 뉴런 2번: "앞은 뚫려있다!" → 1.50
 뉴런 3번: "목적지가 오른쪽이다!" → 3.20
 뉴런 4번: "왼쪽은 막혀있다!" → 0.80
 ...
 뉴런 64번: "뒤쪽은 안전하다!" → 0.30

이 64개의 "상황 분석 결과"가 출력층으로 전달되고,
 출력층이 이걸 종합해서 최종 행동을 결정합니다!

- 출력층 "직진 뉴런": 64개 분석을 보고 → Q(직진) = 8.5 점!
- 출력층 "우회전 뉴런": 64개 분석을 보고 → Q(우회전) = 3.2 점!
- 출력층 "좌회전 뉴런": 64개 분석을 보고 → Q(좌회전) = 2.1 점!
- 직진이 8.5로 가장 높으니까 → "직진!" 을 선택!

왜 이렇게 나누었을까? 병원에 비유하면 바로 이해됩니다.

 비유: 병원 진단

환자가 왔습니다. 증상이 여러 개 있어요.

- 열: 38.5도
- 기침: 있음
- 두통: 약간
- 콧물: 없음

1단계: 검사실 64개가 각각 "이 증상 조합이 무엇을 의미하는지" 분석

검사실 1: "감기 패턴이다!" → 높은 점수

검사실 2: "독감 패턴이다!" → 중간 점수
 검사실 3: "알레르기 패턴이다!" → 낮은 점수

이 단계에서 "약을 먹어라" 같은 처방은 안 합니다!
 그냥 "어떤 병의 패턴인지"만 분석합니다.

2단계: 최종 의사 3명이 검사 결과를 보고 처방을 결정

의사 A (감기약 담당): 64개 검사결과를 보고 → "감기약 필요도: 8.5"
 의사 B (독감약 담당): 64개 검사결과를 보고 → "독감약 필요도: 3.2"
 의사 C (알레르기약 담당): 64개 검사결과를 보고 → "알레르기약 필요도: 2.1"

→ 감기약 점수가 가장 높으니까 → "감기약 처방!"

우리 AI도 같은 구조입니다!

검사실 = 은닉층 64개 뉴런 (상황 분석)
 의사 = 출력층 3개 뉴런 (행동 결정)

자, 다시 정리하면:

왜 전부 더하는가?

× "벽 위치를 더해서 어디로 갈지 계산한다" (이것은 오해!)
 ✓ "여러 정보를 하나의 '감지 점수'로 합친다" (이것이 정답!)

더하기 = 이 뉴런이 맡은 패턴이 "지금 얼마나 해당되는지" 점수를 매기는 것!

벽이 대각선에 있으면 → 높은 점수 → "이 패턴 맞아!"
 벽이 없으면 → 낮은 점수 → "이 패턴 아니야"

3단계: 활성화 함수를 통과시킨다 (ReLU)

$\text{ReLU}(2.86) = 2.86$ ☑ (양수니까 그대로 통과!)

만약 합계가 -1.5였다면?

$\text{ReLU}(-1.5) = 0$ ✕ (음수는 0으로 차단!)

2.86의 의미:

→ 이 뉴런은 "대각선 벽 패턴"을 2.86만큼 감지했다!
 → 이 숫자가 직접 "직진해!" 같은 답은 아님!
 → 64개 뉴런이 각각 다른 패턴을 감지하고,
 출력층이 그 결과를 종합해서 최종 행동을 결정!

이걸 수식으로 쓰면:

뉴런 출력 = $\text{ReLU}(\sum(\text{입력} \times \text{가중치}) + \text{편향})$

가중치(weight) 가 바로 AI가 학습하는 것입니다! 처음에는 가중치가 **완전 랜덤**이라 엉터리 값이 나옵니다. 하지만 학습을 수천 번 반복하면서 "이 가중치를 이렇게 바꾸면 더 정확해지겠다"를 발견하고, 조금씩 수정해나갑니다. 마치 처음에는 아무렇게나 조율된 기타를 수백 번 연습하면서 제 소리가 나게 맞추는 것과 같습니다.

💡 **ReLU는 뭐고, Sigmoid는?** 방금 3단계에서 본 **ReLU**는 "음수는 0으로 차단, 양수는 통과"하는 필터입니다. 쓸모없는 신호(음수)를 걸러내서 중요한 정보만 남기는 역할이에요. 프로그래밍에서 배운 **Sigmoid**는 숫자를 0~1 사이 확률로 바꾸는 함수였죠? 우리 AI의 은닉층에서는 **ReLU**를 씁니다. Sigmoid는 "스팸일 확률 92%"처럼 확률이 필요할 때 사용합니다.

왜 Q값이 4개(전진/후진/위/아래)가 아니라 3개(직진/우회전/좌회전)일까요?

우리 자동차는 "바라보는 방향" 기준으로 움직입니다!

자동차가 오른쪽을 보고 있다면:

직진 = 오른쪽으로 이동
우회전 = 아래쪽으로 이동
좌회전 = 위쪽으로 이동

자동차가 위쪽을 보고 있다면:

직진 = 위쪽으로 이동
우회전 = 오른쪽으로 이동
좌회전 = 왼쪽으로 이동

→ 상/하/좌/우 4방향이 아니라,
"내가 보는 방향 기준으로 직진/우회전/좌회전" 3가지!
→ 후진은 없습니다! (효율을 위해 앞으로만 가도록 설계)

💡 **어렵게 느껴지면 이것만 기억하세요!**

- 뉴런 하나는 "이 상황이 어떤 패턴인지 감지하는 센서" (행동을 결정하지 않음!)
- 가중치는 "어떤 정보를 중요하게 볼지 정하는 숫자" (처음엔 랜덤, 학습하며 수정!)
- 전부 더하는 이유 = 여러 정보를 하나의 "감지 점수"로 합치기 위해서!
- 은닉층 64개 뉴런이 **상황을 분석**하고, 출력층 3개 뉴런이 **행동을 결정**한다!
- 학습 = "가중치를 조금씩 수정해서 더 정확한 Q값을 내도록 하는 것"

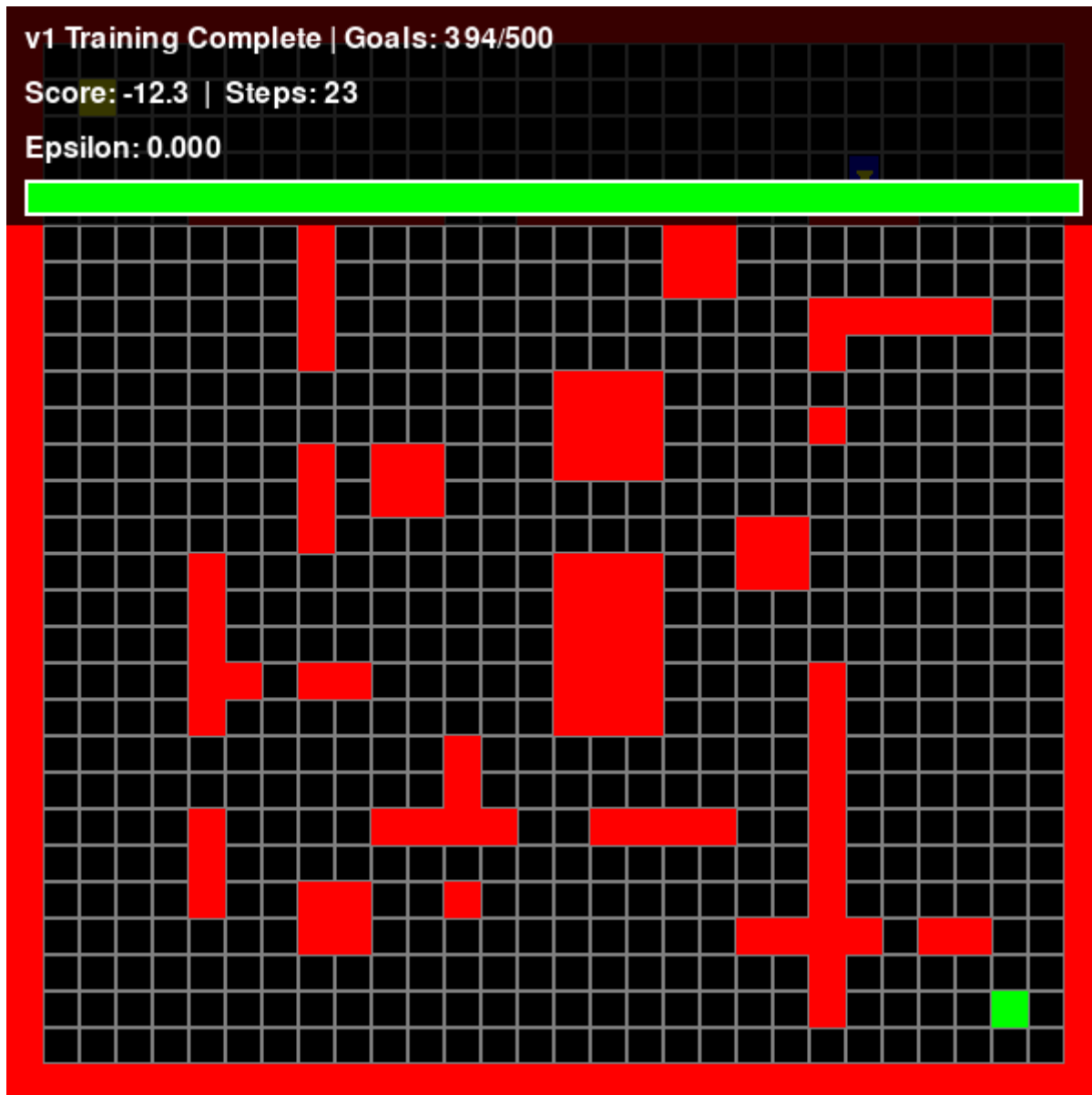
이 과정을 수백, 수천 번 반복하면 Q값이 점점 정확해집니다. 이것이 DQN의 핵심입니다!

📖 **직접 실행해보기!**

💡 **지금 바로 v1을 실행해볼 수 있습니다!**

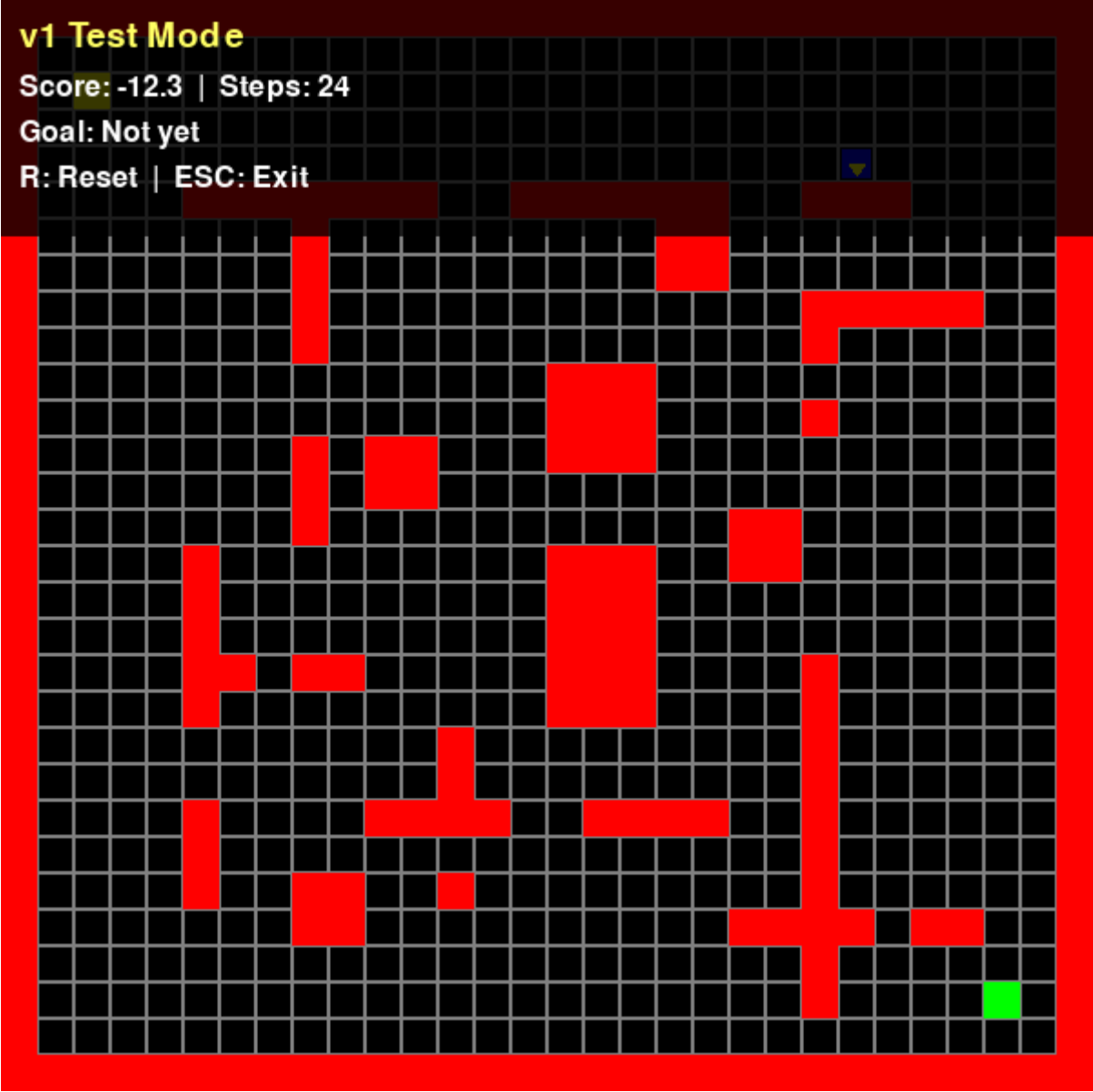
학습 실행 (AI가 스스로 배우는 과정):

```
cd simulator-v1
python train.py
```



학습된 AI로 실행 (배운 것을 활용하는 과정):

```
cd simulator-v1
python main.py
```

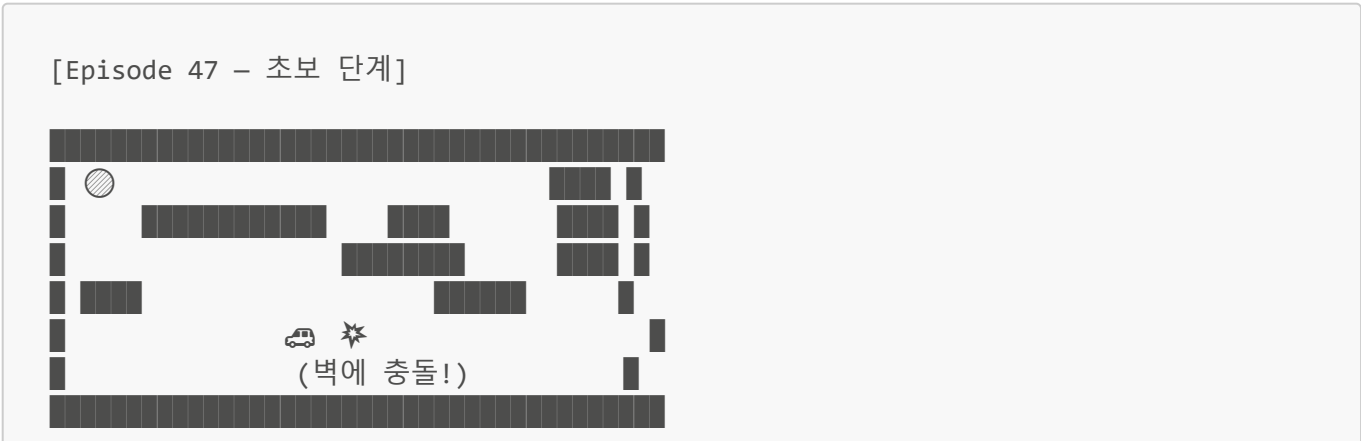


🎮 학습 시작!

드디어 화면에 작은 파란 자동차가 나타났습니다. 메타코딩은 심장이 두근거렸습니다. 내가 만든 AI가 처음으로 세상에 나온 순간!

하지만 처음에는... 처참했습니다.

처음 100번 (Episode 0~100): 아무것도 모르는 갓난아이



Episode: 47 | Score: -10.0 | ϵ : 0.79

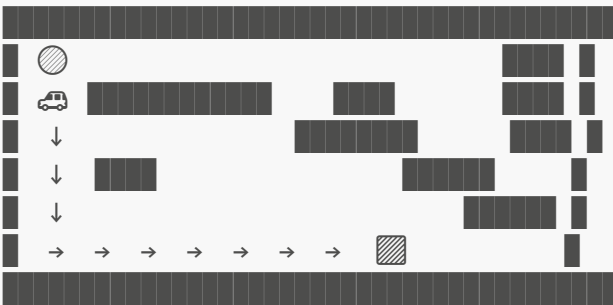
→ 랜덤하게 움직이다 벽에 충돌!

"어... 이게 AI야?"

자동차가 술 취한 person처럼 비틀거리며 벽에 광광 부딪혔습니다. 하지만 이건 정상입니다! ϵ (엡실론) = 0.79 라는 숫자를 보세요. 이것은 "79%의 확률로 랜덤하게 행동한다"는 뜻입니다. 아직은 AI가 세상을 탐험하는 단계입니다. 마치 아기가 처음 걸음마를 시작할 때처럼, 넘어지고 부딪히면서 세상을 알아가는 중이에요.

중간 200번 (Episode 100~300): 뭔가 감을 잡기 시작

[Episode 213 - 학습 중]



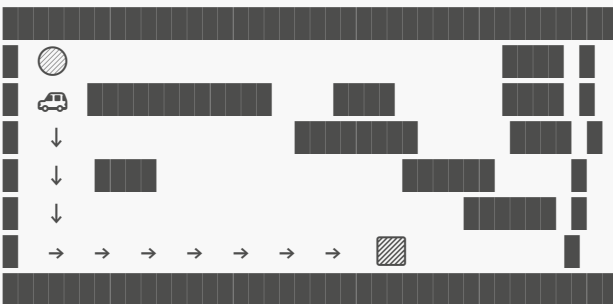
Episode: 213 | Score: 52.3 | ϵ : 0.34

→ 패턴을 찾기 시작! 아래로 가다가 오른쪽으로!

뭔가 달라졌습니다! 자동차가 의미 있는 경로를 따라 움직이기 시작했습니다. 아직 완벽하지 않지만, "아래로 가다가 오른쪽으로"라는 패턴을 발견한 것이죠. ϵ 도 0.34로 줄어들었습니다. "34%만 탐험, 66%는 배운 대로 행동." 점점 배운 것을 활용하기 시작한 겁니다.

마지막 200번 (Episode 300~500): 완성!

[Episode 487 - 완성 단계]



Episode: 487 | Score: 98.2 | ϵ : 0.08

→ 매번 같은 경로로 성공! 성공률 80%!

"와! 진짜 된다! AI가 스스로 길을 찾았어!"

메타코딩은 의자에서 벌떡 일어나 주먹을 불끈 쥐었습니다. 500번의 시행착오를 통해 AI가 스스로 최적 경로를 발견한 것입니다. 이 기쁨은 직접 만들어본 사람만 알 수 있습니다.

🗺️ 그런데... 밤늦게까지 보다 보니

흥분이 가라앉고 나서, 메타코딩은 이상한 점을 발견했습니다.

자동차가 **항상 똑같은 경로**로만 움직였습니다.

매번 이것만 반복:
시작점(●) → 아래로 쪽 → 오른쪽으로 쪽 → 목적지(■)

"뭐, 도착하니까 괜찮지? 다른 길이 있어도 이 길로 가면 되잖아."

메타코딩은 스스로를 설득하고 잠자리에 들었습니다.

★ 다음날 — 팀장님 앞에서 대참사

"자, 팀장님! 보여드리겠습니다!"

자신 있게 프로그램을 실행했습니다. 자동차가 익숙한 경로로 목적지에 도달했습니다. 팀장님도 "오!" 하며 고개를 끄덕였습니다.

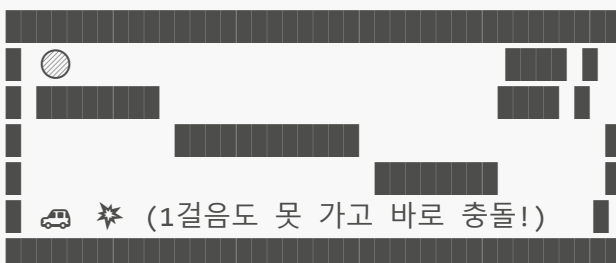
"좋네요! 그런데 메타코딩 씨, 청소 로봇은 여러 집에서 작동해야 하잖아요. 이 맵이 아닌 **다른 맵**에서도 잘 되나요?"

팀장님이 설정을 바꿨습니다.

```
env = GridEnvironment(random_map=True) # 새로운 맵!
```

결과:

[새로운 맵에서 v1 실행]



→ 새로운 맵에서는 완전히 무력!

메타코딩의 얼굴이 새빨개졌습니다.

팀장님: "메타코딩 씨... 이 맵에서만 작동하는 거예요?"

메타코딩: "아... 네... 죄송합니다..."

🤖 무엇이 문제였을까?

집에 돌아와서 메타코딩은 문제를 분석했습니다.

문제 1: 항상 같은 방향에서 시작

```
# car.py
self.direction = 0 # 항상 위쪽만 보고 시작!
```

항상 위쪽을 보고 시작하니까, AI는 "위에서 시작하면 아래로 내려가라"만 배운 것입니다. 오른쪽이나 왼쪽을 보고 시작하면 어떻게 해야 할지 전혀 모릅니다.

문제 2: 항상 같은 맵에서 학습

```
# train.py
env = GridEnvironment() # 항상 같은 맵!
```

한 집의 구조만 외운 청소부가 다른 집에 가면? 당연히 길을 잃습니다.

문제 3: 너무 빨리 탐험을 멈춤

```
self.epsilon_decay = 0.995 # 빠르게 탐험 줄임
```

학습 과정에서 ϵ (엡실론) = 0.79 같은 숫자를 봤죠? 이건 "79% 확률로 랜덤하게 행동"한다는 뜻입니다. AI에게는 두 가지 모드가 있습니다:

- **탐험(Exploration):** 새로운 것을 시도해보기 (랜덤 행동)
- **활용(Exploitation):** 배운 것을 써먹기 (Q값이 높은 행동)

처음에는 아무것도 모르니까 탐험을 많이 해야 하고, 배움이 쌓이면 점점 활용으로 넘어가야 합니다. 이 전환 속도를 조절하는 게 `epsilon_decay`입니다.

v1의 문제는 이 전환이 너무 빨랐다는 것! 세상의 10%도 안 봤는데 "이제 아는 길로만 가자!"라며 탐험을 포기해버린 겁니다. 마치 식당 세 군데만 가보고 "여기가 최고!"라며 다른 곳을 안 가본 것과 같습니다.

💡 0.995 vs 0.998의 차이가 얼마나 큰지는 2화에서 자세히 비교합니다!

이 세 가지 문제가 합쳐진 결과가 바로 **과적합(Overfitting)**입니다.

과적합이란 ***연습 문제는 완벽한데, 새 문제는 전혀 못 푸는 것***입니다. 학생이 수학 문제집 1번만 1000번 반복해서 눈 감고도 풀 수 있게 됐는데, 시험에 2번이 나오면 완전히 틀리는 것과 같습니다. 원리를 이해한 게 아니라 **답을 외운 것**이니까요.

반대 개념은 **일반화(Generalization)** — "어떤 문제가 나와도 풀 수 있는 진짜 실력"입니다. 3화에서 이 과적합을 어떻게 **발견하고 해결하는지** 자세히 다룹니다.

메타코딩은 노트에 크게 적었습니다.

"v1의 교훈: 과적합은 가짜 성공이다. 진짜 실력은 새로운 상황에서 드러난다."

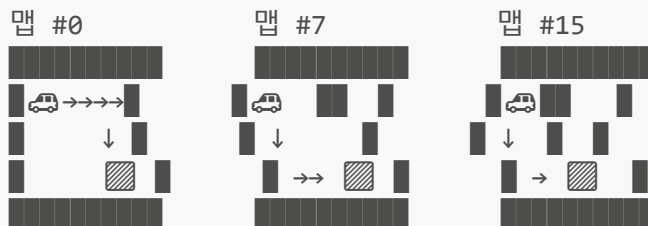
"내일부터 다시 시작이다!"

2화: 넘어져야 걷는 법을 안다 (v2)

📍 지금 배우는 것 → 경사하강법, 역전파 (딥러닝의 학습 원리)
지도학습 | 비지도학습 | 강화학습 ★ | 딥러닝 ★

🔧 2화에서 만드는 것: simulator-v2

20개 맵에서 돌아가며 학습!



특징: 3000 에피소드, 매 150번마다 맵 교체
성공률: 70% (처음 보는 맵에서도!)

📅 2주차 월요일 — 다시 시작

주말 내내 v1의 실패가 머릿속에서 떠나지 않았습니다. 월요일 아침, 메타코딩은 커피를 한 잔 크게 내리고 노트를 펼쳤습니다.

"문제는 세 가지. 같은 방향, 같은 맵, 빠른 탐험 종료. 하나씩 해결하자."

🔑 해결책 1: 랜덤 방향으로 시작

```
# v1 (항상 위쪽을 봄)
self.direction = 0

# v2 (랜덤하게 시작!)
self.direction = random.randint(0, 3)
# 0: 위, 1: 오른쪽, 2: 아래, 3: 왼쪽
```


이제 매 에피소드마다 다른 방향으로 시작합니다.

Episode 1: 위쪽 보고 시작 → 위로 가는 법 배움
 Episode 2: 오른쪽 보고 시작 → 오른쪽으로 가는 법 배움
 Episode 3: 아래쪽 보고 시작 → 아래로 돌아가는 법 배움
 Episode 4: 왼쪽 보고 시작 → 왼쪽에서 시작하는 법 배움
 ...
 → 어떤 방향에서 시작해도 대응 가능! ☑

🔑 해결책 2: 여러 맵에서 학습

```
# 150 에피소드마다 맵 변경
if episode % 150 == 0:
    map_id = random.randint(0, 19) # 0~19번 맵 중 하나
    env.reset_map(map_id)
```

20개의 다양한 맵을 돌아가며 학습합니다.

맵 0 (빈 맵) : 직진하는 법 배움
 맵 5 (중간 맵) : 장애물 하나 피하는 법 배움
 맵 10 (복잡한 맵) : 복잡한 미로 탈출하는 법 배움
 맵 15 (좁은 통로) : 좁은 길을 따라가는 법 배움
 → 어떤 환경에서도 어느 정도 작동! ☑

여러 문제집을 번갈아 풀면 진짜 수학 실력이 느는 것처럼, AI도 다양한 환경에서 학습해야 진짜 실력이 붙습니다.

🧠 경사하강법(Gradient Descent): AI가 실수에서 배우는 원리

한 마디로: AI가 "틀렸네? 다음엔 이쪽으로 수정하자!"를 반복하는 것! 산에서 가장 낮은 곳을 찾아 한 걸음씩 내려가는 것과 같습니다.

메타코딩: "AI가 스스로 배운다는 건 알겠는데, **정확히 어떻게** 배우는 거지?"

김선배: "간단해요. AI가 **틀릴 때마다, 틀린 방향의 반대로 조금씩 수정**하는 거예요."

이것이 바로 **경사하강법(Gradient Descent)** 입니다. 이름은 어렵지만, 원리는 놀라울 정도로 직관적이에요.

☁️ 비유: 안개 속 산에서 가장 낮은 곳 찾기

여러 봉우리와 골짜기가 있는 복잡한 산을 상상해보세요. 짙은 안개가 끼어서 주변이 하나도 안 보입니다. 전체 지도도 없습니다. 이 상황에서 ****가장 낮은 곳(골짜기)****을 찾아야 합니다.

어떻게 하면 될까요?

전략: 발로 경사를 느끼면서 내려가기!

1. 발을 앞뒤좌우로 내밀어서 경사를 느낀다 → "기울기 계산"
2. 가장 가파르게 내려가는 방향으로 한 걸음 → "가중치 업데이트"
3. 다시 경사를 느끼고, 또 한 걸음 → "반복!"
4. 더 이상 내려갈 곳이 없으면 멈춘다 → "수렴"

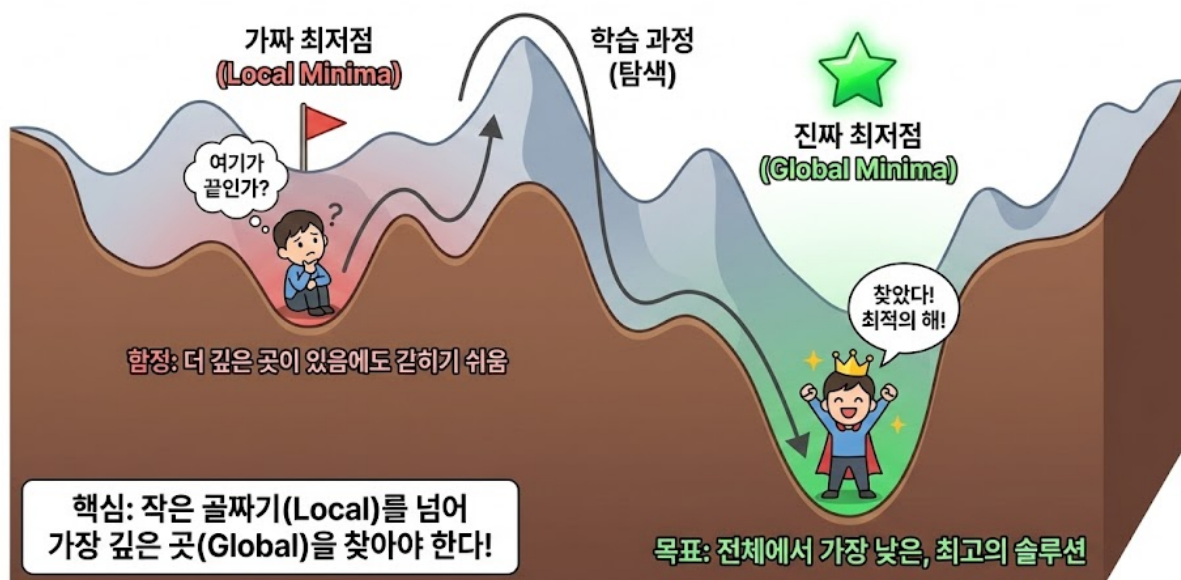
이것이 경사하강법의 전부입니다! 생각보다 간단하죠?

여기서 "경사를 느끼는 것"이 바로 미분(기울기 계산)입니다. 수학의 미분이 여기서 쓰입니다. AI가 "내가 지금 있는 위치에서 어느 방향으로 가면 오차가 줄어들까?"를 계산하는 것이 미분이에요.

☝ 하지만 함정이 있습니다: 로컬 미니마!

산에는 여러 골짜기가 있습니다. 발로만 느끼면서 내려가다 보면 ***가짜 골짜기***에 빠질 수 있습니다.

로컬 미니마 vs 글로벌 미니마: 최적화의 함정과 목표



"가짜 골짜기(로컬 미니마)"에 도달하면, 발로 느껴보는 모든 방향이 올라가는 것 같습니다. "여기가 가장 낮은 곳이구나!"라고 착각하죠. 하지만 실제로는 더 낮은 곳(진짜 최저점, 글로벌 미니마)이 멀리 있습니다.

🦶 보폭(Learning Rate)이 중요한 이유

안개 속에서 한 걸음의 크기를 보폭(Learning Rate) 이라고 합니다. 이 보폭이 학습의 운명을 결정합니다.

보폭(Learning Rate)에 따른 3가지 결과:

[보폭이 너무 클 때]

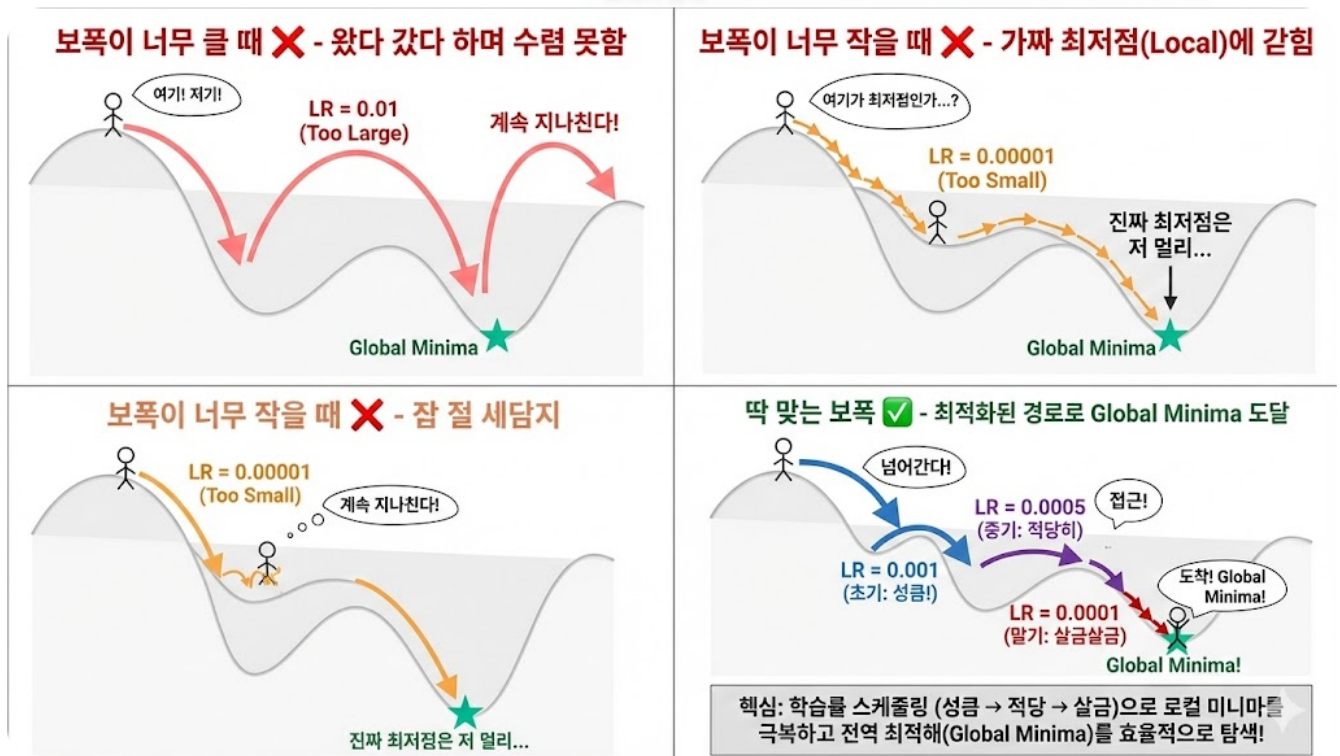
- 최저점을 지나쳐서 반대편 산으로! →→→→→→● 또 지나침!
- 계속 산을 넘나들며 수렴하지 못함 ✕

[보폭이 너무 작을 때]

- 가짜 골짜기(로컬 미니마)에 빠짐!
- 이 산을 넘을 에너지가 없어서 진짜 최저점을 못 찾음 ✕

[딱 맞는 보폭]

- 로컬 미니마를 뛰어넘고 →→● 진짜 최저점 도달! ✓
- 처음엔 크게 훑고, 나중엔 작게 정밀 탐색!



🔗 최적의 전략: 처음엔 크게, 나중엔 작게!

가장 똑똑한 방법은 보폭을 점점 줄이는 것입니다.

- 1단계: 큰 보폭으로 전체를 훑는다
 "이 산 어디쯤에 가장 낮은 곳이 있을까?"
 → 성큼성큼 걸어서 대략적인 위치를 파악!
 → 로컬 미니마도 큰 보폭으로 뛰어넘을 수 있음!
 - 2단계: 작은 보폭으로 정밀하게 찾는다
 "이 근처가 가장 낮은 것 같은데, 정확히 어디지?"
 → 조심조심 걸어서 진짜 최저점을 찾음!
- 빠르면서도 정확! ✓
- 이 기법을 Learning Rate Scheduling이라 하며, 4화에서 자세히 다룹니다!

📌 실제 AI에서 경사하강법은 이렇게 동작합니다

추상적인 산 이야기에서 구체적인 숫자로 내려와 봅시다.

[v2에서 한 번의 학습 과정 - 6단계]

1단계: AI가 Q값을 예측한다

현재 상황 → 신경망 통과 → Q(직진) = 7.0
 "직진하면 7.0점짜리 좋은 결과가 나올 거야!"

2단계: 실제로 행동하고 결과를 확인한다

직진했더니 → 목적지에 가까워짐!
 실제 보상 기반 올바른 Q값(정답): 10.0

3단계: 오차(Loss)를 계산한다

오차 = (예측 - 정답)²
 = (7.0 - 10.0)²
 = (-3.0)²
 = 9.0
 → "꽤 많이 틀렸네!"

이 오차를 손실(Loss) 이라고 합니다.
 손실이 산의 높이입니다. 이걸 줄이는 게 목표!

4단계: 기울기(Gradient)를 계산한다 - 이것이 미분!

기울기 = 2 × (예측 - 정답)
 = 2 × (7.0 - 10.0)
 = -6.0

기울기가 음수(-6.0) → "Q값을 올려야 해!"
 기울기가 양수였다면 → "Q값을 내려야 해!"

이 기울기가 "경사"입니다.
 "어느 방향으로 가중치를 수정하면 오차가 줄어드는가?"
 를 알려주는 나침반 같은 존재!

5단계: 가중치를 수정한다 (한 걸음 내딛기!)

학습률(보폭) = 0.1
 수정량 = 학습률 × |기울기| = 0.1 × 6.0 = 0.6

새 Q값 = 7.0 + 0.6 = 7.6
 → "10.0에 조금 가까워졌어!"

6단계: 이걸 3000번 반복!

7.0 → 7.6 → 8.1 → 8.5 → 9.0 → 9.5 → 9.8 → 10.0

→ 점점 정답에 가까워짐! ☒

💡 어렵게 느껴지면 이것만 기억하세요! 경사하강법을 한 문장으로 요약하면: "AI가 예측하고 → 틀리면 → 틀린 만큼 수정하고 → 이걸 수천 번 반복하면 → 정답에 가까워진다!" 숫자 계산은 컴퓨터가 알아서 합니다. 우리는 원리만 이해하면 돼요!

📦 역전파(Backpropagation): 오차를 거꾸로 추적하기

방금 "기울기를 계산한다"고 했는데, 신경망에는 뉴런이 64개나 있습니다. 각 뉴런의 가중치를 어떻게 수정할까요?

메타코딩: "뉴런이 64개나 되는데, 어떤 뉴런의 가중치를 얼마나 바꿔야 하는 거지?"

답은 **역전파(Backpropagation)** 입니다.

핵심 원리:

1. 은닉층의 뉴런들이 각자 계산을 하면서 값을 바꿔가고 → 최종 출력(Q값)이 나옵니다
2. 그 Q값이 맞는지 실제 결과와 비교합니다 → 틀렸다!
3. ***"어떤 뉴런이 얼마나 영향을 줘서 틀린 결과가 나왔는지"***를 거꾸로 추적합니다
4. 영향을 많이 준 뉴런의 가중치는 많이 고치고, 적게 준 뉴런은 조금만 고칩니다

쉬운 예시로 보겠습니다:

[역전파 예시 - 간단한 3뉴런 신경망]

입력: "앞에 벽 없음, 목적지가 오른쪽"

뉴런A (벽 감지) —가중치 0.7—
 뉴런B (방향 감지) —가중치 0.3—

출력 뉴런 → Q(직진) = 7.0

실제로 직진했더니 → 목적지에 도착! → 정답 Q값은 10.0이었어야 함!

오차 = 10.0 - 7.0 = 3.0 (3점이나 부족했음! Q값을 올려야 함!)

역전파 시작! "누가 더 책임이 크지?"

학습률(learning rate) = 0.1 이라고 합시다.
 (학습률 = "한 번에 얼마나 고칠지"를 정하는 보폭)

수정량 공식: 오차 × 학습률 × 그 뉴런의 책임 비율

뉴런A의 가중치 = 0.7 → 출력에 70% 영향
 → 수정량 = 오차(3.0) × 학습률(0.1) × 책임비율(0.7)
 = 3.0 × 0.1 × 0.7
 = 0.21
 → 새 가중치: 0.7 + 0.21 = 0.91 ☒

뉴런B의 가중치 = 0.3 → 출력에 30% 영향
 → 수정량 = 오차(3.0) × 학습률(0.1) × 책임비율(0.3)
 = 3.0 × 0.1 × 0.3
 = 0.09
 → 새 가중치: 0.3 + 0.09 = 0.39 ☑

정리:

0.21과 0.09는 그냥 예시가 아닙니다!
 "오차 × 학습률 × 책임비율"로 계산된 값입니다!

- 영향을 많이 준 뉴런A = 책임이 크다 = 0.21만큼 크게 수정!
- 영향을 적게 준 뉴런B = 책임이 작다 = 0.09만큼 조금 수정!
- 다음번에 같은 상황이 오면 Q(직진) = 7.0보다 높게 나옴!
- 이걸 수천 번 반복하면 Q값이 점점 정확해짐!

[역전파의 전체 흐름]

순전파 (앞으로 → 결과 계산):

입력(11개) → 은닉층(64개 뉴런) → 출력(Q값 3개)
 ↓
 "Q(직진) = 7.0"
 ↓
 실제 행동 → 결과 확인
 ↓
 "정답은 10.0이었어!"
 ↓
 오차 = 7.0 - 10.0 = -3.0

역전파 (거꾸로 ← 책임 추적):

출력 오차(-3.0) → 출력층 가중치 수정
 → 은닉층 각 뉴런에게 "네 책임은 이만큼!"
 → 각 뉴런의 가중치 수정
 → 입력층 가중치도 수정

- 결과에 가까운 뉴런 = 책임이 크다 = 많이 수정!
- 결과에서 먼 뉴런 = 책임이 작다 = 조금만 수정!

🔗 연쇄법칙(Chain Rule): 역전파의 비밀 무기

역전파가 "결과에서 거꾸로 돌아가면서 각 가중치를 조정한다"는 건 이해했죠? 그런데 "각 가중치를 얼마만큼 조정해야 하는 걸까?" 이걸 계산하는 공식이 연쇄법칙입니다.

전화 게임(귓속말 릴레이)으로 이해해봅시다.

[상민이(a)] → [지유(b)] → [현우(c)] → [최종 발표(y)]

최종 발표에서 "사과"라고 해야 하는데 "바과"라고 틀렸어요!

핵심: 결과에서 가까울수록 영향력이 크다!

현우(c): 바로 앞 → 영향력 0.5 (가장 큼! → 가중치 많이 수정!)

지유(b): 현우를 거침 → $0.3 \times 0.5 = 0.15$ (줄어듦 → 조금 수정)

상민이(a): 둘 다 거침 → $0.4 \times 0.3 \times 0.5 = 0.06$ (아주 작음 → 살짝 수정)

→ 멀수록 곱셈이 쌓여서 영향력이 줄어든다!

→ 마치 도미노가 넘어질 때 멀리 있을수록 충격이 줄어드는 것!

왜 ***연쇄(Chain)***법칙이냐고요? 앞 단계의 결과가 **사슬처럼 연결**되어서 다음 계산에 쓰이기 때문입니다.

🔑 해결책 3: 천천히 탐험 줄이기 (Epsilon 조절)

Epsilon(ϵ) 은 "랜덤하게 새로운 것을 시도할 확률"입니다.

$\epsilon = 1.0$ (100%): "모험가 모드! 완전 랜덤으로 움직여!"

$\epsilon = 0.5$ (50%): "절반은 모험, 절반은 아는 길로"

$\epsilon = 0.1$ (10%): "거의 아는 길, 가끔 새 시도"

$\epsilon = 0.0$ (0%): "베테랑 모드! 배운 것만 활용!"

v1 (너무 빨리 탐험을 줄임)

self.epsilon_decay = 0.995 # 금방 0에 가까워짐

v2 (천천히 탐험을 줄임)

self.epsilon_decay = 0.998 # 오래 탐험 유지

왜 이게 중요할까요?

v1의 문제:

Episode 1 : $\epsilon = 1.00$ (100% 탐험) "뭐든 해봐!"

Episode 50 : $\epsilon = 0.60$ (60% 탐험) "좀 알아가는 중"

Episode 100 : $\epsilon = 0.40$ (40% 탐험) "이제 아는 길로"

Episode 200 : $\epsilon = 0.10$ (10% 탐험) "거의 아는 길만"

→ 세상의 10%도 안 봤는데 탐험을 포기!

→ 한 가지 길만 알고 멈춤!

v2의 개선:

Episode 100 : $\epsilon = 0.90$ (90% 탐험) "아직 많이 탐험"

Episode 500 : $\epsilon = 0.60$ (60% 탐험) "여전히 탐험 중"

Episode 1000 : $\epsilon = 0.40$ (40% 탐험) "배워가는 중"

Episode 2000 : $\epsilon = 0.10$ (10% 탐험) "이제 활용 시작"

→ 충분히 탐험한 후에 활용! ☑

🧠 탐험과 활용의 균형을 잡다

1화에서 ****탐험(Exploration)****과 **활용(Exploitation)** 개념을 간단히 소개했죠? 이 균형이 왜 중요한지 좀 더 자세히 봅시다.

여러분도 이런 경험 있지 않나요? "새로운 식당을 가볼까, 맨날 가는 단골 식당에 갈까?"

탐험만 하면 → 매일 새 식당인데 맛없을 수도 있음 (배운 것을 써먹지 못함)
 활용만 하면 → 단골집만 가니까 더 맛있는 곳을 영영 모름

최적의 균형:

처음 한 달은 이것저것 먹어보고 (Epsilon 높음)
 그 후엔 맛있던 곳 위주로 가되, 가끔 새 곳도 시도 (Epsilon 낮음)

v1은 이 균형을 잘못 잡은 것입니다. 식당 세 군데만 가보고 "여기가 최고!"라며 다른 곳을 안 가본 셈이죠. v2에서는 충분히 다양한 식당을 돌아본 후에 단골집을 정합니다.

🔧 해결책 4 & 5: 추가 개선들

새로운 길 발견 보상:

```
# 처음 가보는 곳이면 보너스!
if (next_x, next_y) not in self.visited_positions:
    reward += 0.2
```

같은 길만 빙빙 도는 것을 방지합니다. 새로운 칸을 밟을 때마다 "잘했어!"라는 작은 보상을 줘서 탐험을 격려하는 것이죠.

학습 안정화 (Gradient Clipping):

```
# 한 번에 너무 크게 배우지 않도록 제한!
torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)
```

Gradient Clipping 비유:

❌ 없을 때:
 어떤 날은 기울기가 100, 어떤 날은 0.001
 → 학습이 불안정, 폭주할 위험!

☑️ 있을 때:
 기울기가 아무리 커도 1.0으로 제한
 → 매일 일정한 보폭으로 꾸준히 학습!

운전에 비유하면:

급커브에서 갑자기 핸들을 확 꺾으면 위험하잖아요.
 "핸들은 최대 여기까지만!" 이라는 제한을 두는 것.

🔄 경험 리플레이(Experience Replay): AI의 복습 노트

한 마디로: AI가 겪은 일을 "일기장"에 적어두고, 나중에 랜덤으로 꺼내 복습하는 것!

v2에서 한 가지 더 중요한 기법이 있습니다. **경험 리플레이**입니다.

비유를 들어봅시다. **요리를 배우는 학생**을 상상해보세요!

[경험 리플레이 비유 – 요리 일기장]

요리 학생이 매일 요리를 하고 "요리 일기장"에 기록합니다:

월요일: "소금 1스푼 넣었더니 → 째다! (실패)"

화요일: "소금 0.5스푼 넣었더니 → 딱 맞았다! (성공)"

수요일: "센 불에 3분 볶았더니 → 탔다! (실패)"

목요일: "중불에 5분 볶았더니 → 맛있다! (성공)"

...100일치 기록

복습 시간!

→ 일기장에서 랜덤으로 5개를 꺼내서 복습!

"아, 소금은 0.5스푼이 맞지!"

"센 불은 위험하지!"

왜 순서대로가 아니라 랜덤으로 꺼낼까?

→ 순서대로 복습하면:

"최근에 배운 건 기억나는데,

처음에 배운 건 까먹었어!"

→ 랜덤으로 복습하면:

"오래 전 경험도 골고루 떠올리니까

전체 요리 실력이 향상!"

AI도 마찬가지입니다! AI는 움직일 때마다 경험을 저장합니다:

경험 = (현재 상태, 행동, 보상, 다음 상태)

예시 (요리 일기장처럼!):

경험 1: ("앞에 벽", "직진", -10, "충돌!") → "앞에 벽일 때 직진하면 안 돼!"

경험 2: ("길이 열림", "직진", +0.5, "가까워짐") → "길 열리면 직진 좋아!"

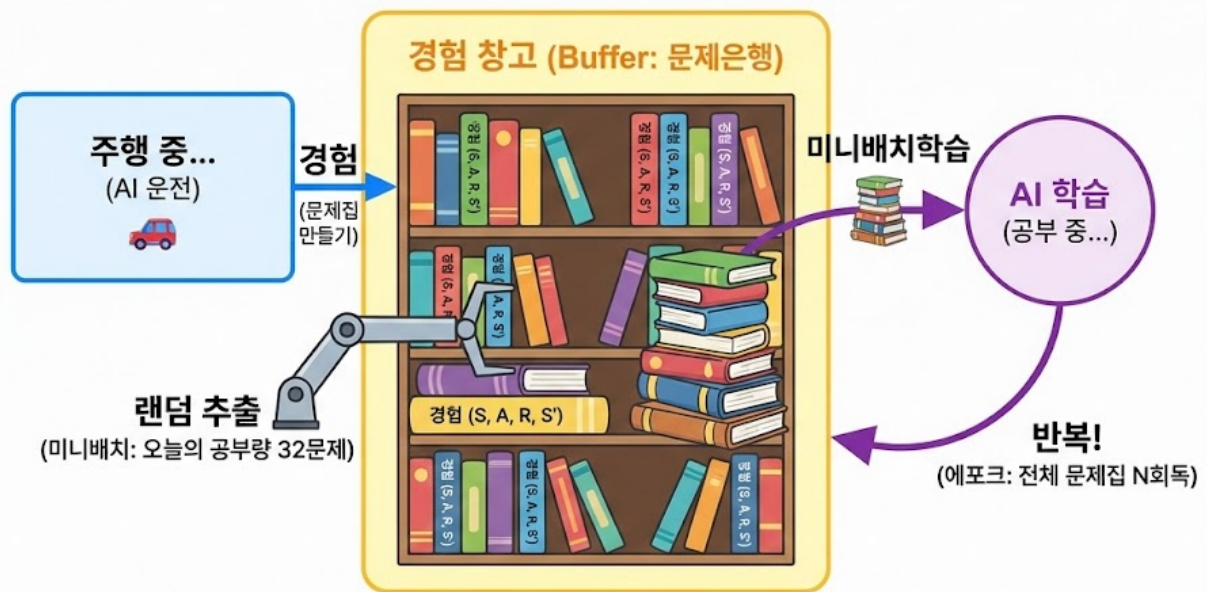
경험 3: ("벽이 오른쪽", "좌회전", +0.5, "안전") → "벽 반대로 가면 좋아!"

...수천 개의 경험이 쌓임!

학습할 때 → 경험 창고에서 랜덤으로 32개를 뽑아서 복습!

→ 옛날 경험도, 최근 경험도, 성공도, 실패도 골고루!

경험 리플레이 (Experience Replay)



경험 리플레이: 과거의 경험(문제)을 ‘문제은행’에 쌓아두고, 시험 공부하듯 매번 랜덤으로 한 묶음(미니배치)씩 뽑아 여러 번 반복(에포크) 학습하여 편식 없이 골고루 배웁니다.

💡 **미니배치(Mini-batch):** 경험 창고에서 한 번에 32개씩 뽑아서 학습하는 것을 **미니배치 학습**이라고 합니다. 왜 하나씩 안 하고 32개씩? 하나씩 하면 학습이 불안정하고(한 경험에 너무 좌우됨), 전부 다 하면 너무 느립니다. 32개는 "적당히 안정적이면서 적당히 빠른" 절충안입니다.

📖 학습 단위 용어 정리

여기서 잠깐, AI 세계에서 자주 쓰는 학습 단위를 정리해봅시다. 비슷한 단어가 여러 개라 헷갈릴 수 있거든요!

학습 단위 정리:

에피소드(Episode): 시작 → 목적지 도달(또는 실패)까지 한 판

→ 우리 자율주행에서 사용!

→ "에피소드 500번 학습" = 500판 플레이!

에포크(Epoch): 전체 데이터를 한 바퀴 돌린 것

→ 이미지 분류 같은 지도학습에서 사용!

→ "10 에포크 학습" = 전체 데이터를 10번 반복!

이터레이션(Iteration): 미니배치 하나를 학습한 한 번

→ 우리 AI에서 경험 32개 뽑아서 학습하는 한 번!

→ 1 에피소드 안에 여러 번의 이터레이션이 있음

쉽게 비유하면:

에피소드 = 시험 1회

에포크 = 문제집 1회독

이터레이션 = 문제 1번 풀기

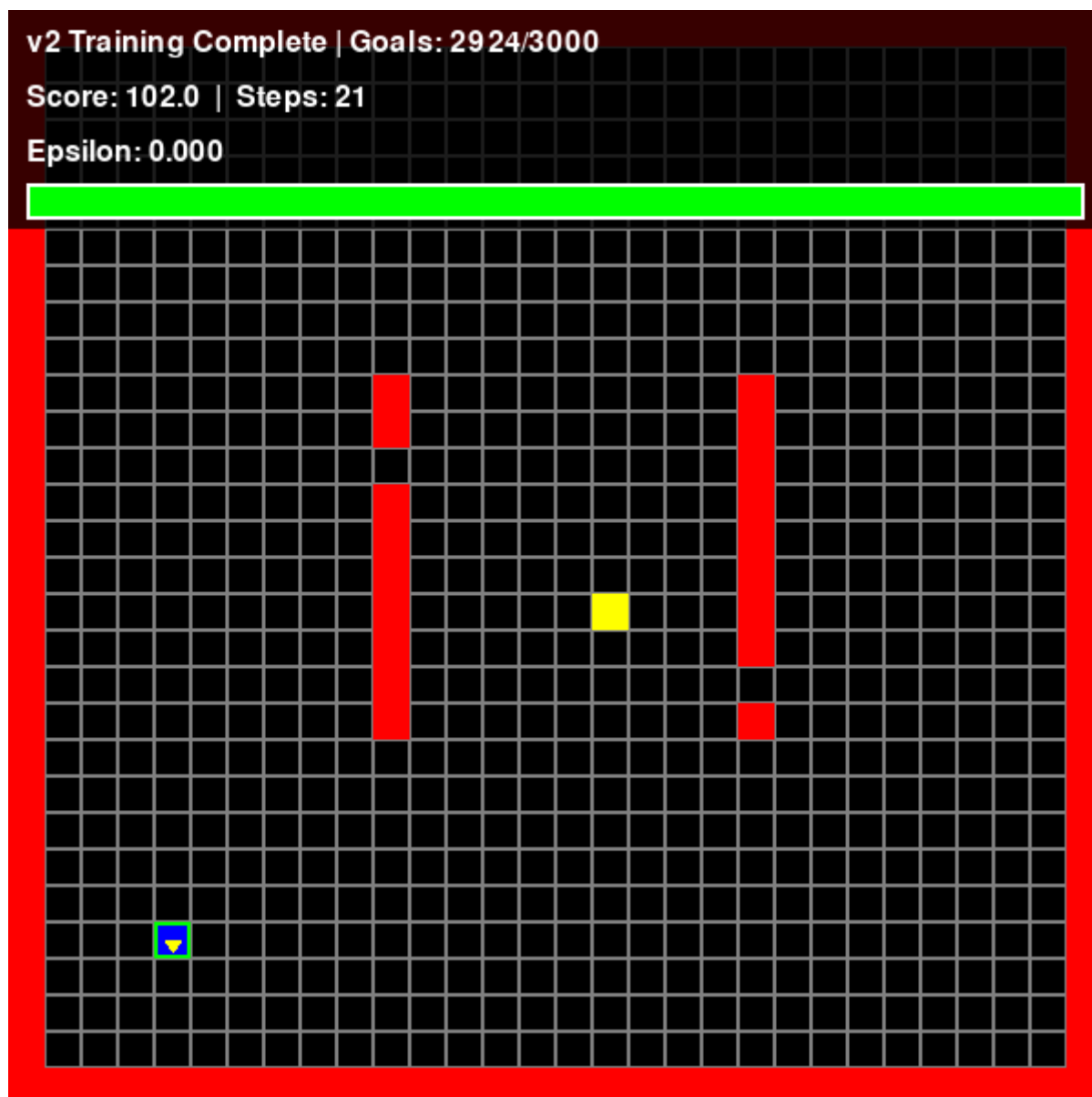
💡 우리 책에서는 주로 "에피소드"를 씁니다. 강화학습은 "에피소드" 단위로 학습하고, 지도학습은 "에포크" 단위로 학습한다는 것만 기억하면 됩니다!

📖 직접 실행해보기!

💡 지금 바로 v2를 실행해볼 수 있습니다!

학습 실행 (20개 맵에서 돌아가며 학습):

```
cd simulator-v2
python train.py
```



학습된 AI로 실행:

```
cd simulator-v2
python main.py
```



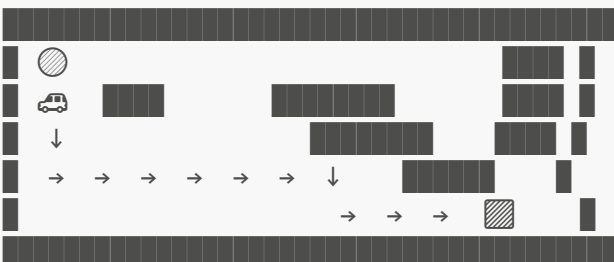
💡 학습(train.py)은 3000 에피소드라 시간이 걸립니다. `config.py`에서 속도를 조절해보세요! (설정 방법은 "시작하기 전에" 참고)

🎮 v2 실행 결과

3000번의 에피소드 동안 20개 맵을 돌아가며 학습합니다.

[v2 - 학습 중 (Episode 1500)]

맵 #7에서 학습 중:

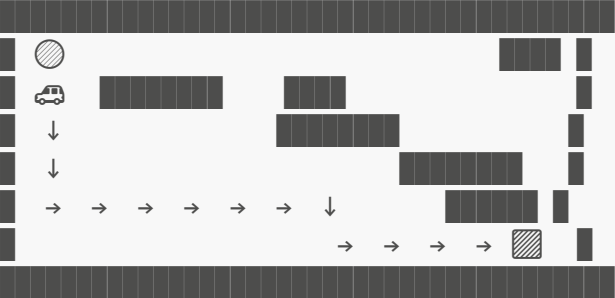


Episode: 1500 | Score: 5.8 | ϵ : 0.47 | Map: #7

3000번의 에피소드가 끝나자, 처음 보는 맵으로 테스트했습니다.

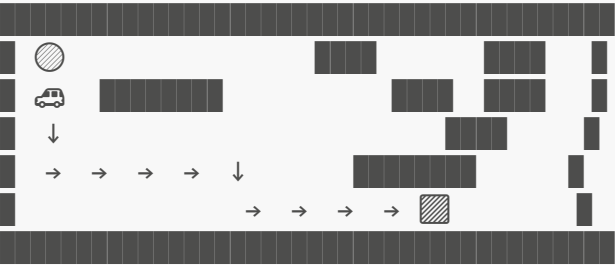
[v2 테스트 – 처음 보는 맵에서]

맵 A (본 적 없는 맵):



☑ 성공!

맵 B (더 복잡한 맵):



☑ 성공!

Episode: 3000 | Success Rate: 70%

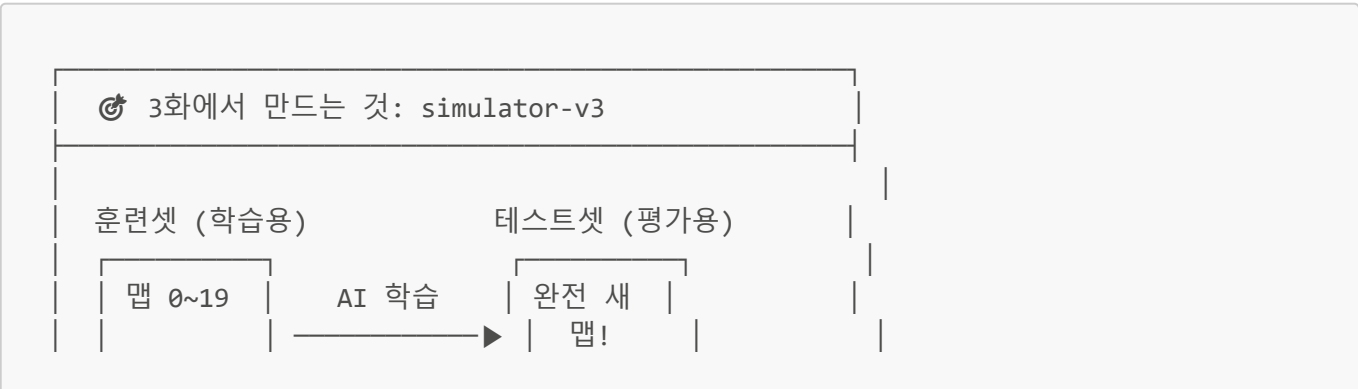
이번엔 팀장님 앞에서도 당당했습니다!

팀장님: "오! 이번엔 다른 맵에서도 잘 작동하네요!"

메타코딩은 속으로 v1의 실패에 감사했습니다. 그 실패가 없었다면 과적합의 무서움을 몰랐을 테니까요.

그런데...

3화: 가짜 90점의 진실 (v3)



(20개)

(처음 봄)

v2 결과 재발견:

같은 맵 테스트 → 90% (부풀려진 성적!)

새로운 맵 테스트 → 68% (진짜 실력!)

♀ 지금 배우는 것 → 모델 평가 (모든 AI 공통!)
 지도학습 ★ | 비지도학습 ★ | 강화학습 ★ | 딥러닝
 (훈련셋/테스트셋, 과적합은 모든 AI에 공통!)

📅 3주차 — 선배의 한 마디

팀장님께 v2를 보여드리고 칭찬을 받은 그날 오후, 메타코딩은 회사 카페에서 커피를 내리고 있었습니다. 그때 뒤에서 익숙한 목소리가 들렸습니다.

"아, 메타코딩! 요즘 강화학습 프로젝트 하고 있다면서?"

김선배였습니다. 3년차 AI 개발자로, 회사에서 머신러닝 관련 질문이 생기면 모두가 찾아가는 사람이었습니다.

"네, v2까지 만들었어요! 이제 다른 맵에서도 70%나 성공해요!"

메타코딩은 자랑스럽게 대답했습니다.

"70%요? 괜찮네요." 김선배가 커피를 받아들며 물었습니다. "근데 그 70%, 어떤 맵으로 테스트한 거예요?"

"20개 맵으로 학습하고, 그 20개 중에서 테스트했어요."

커피를 마시던 김선배가 멈칫했습니다. 눈이 동그아졌습니다.

"잠깐. 학습한 맵이랑 테스트한 맵이 **같다고요?**"

"네... 왜요?"

김선배가 커피를 테이블에 내려놓으며 의자를 끌어당겼습니다.

"메타코딩 씨, 앉아봐요. 이건 중요한 얘이에요."

메타코딩은 갑자기 불안해지며 맞은편에 앉았습니다.

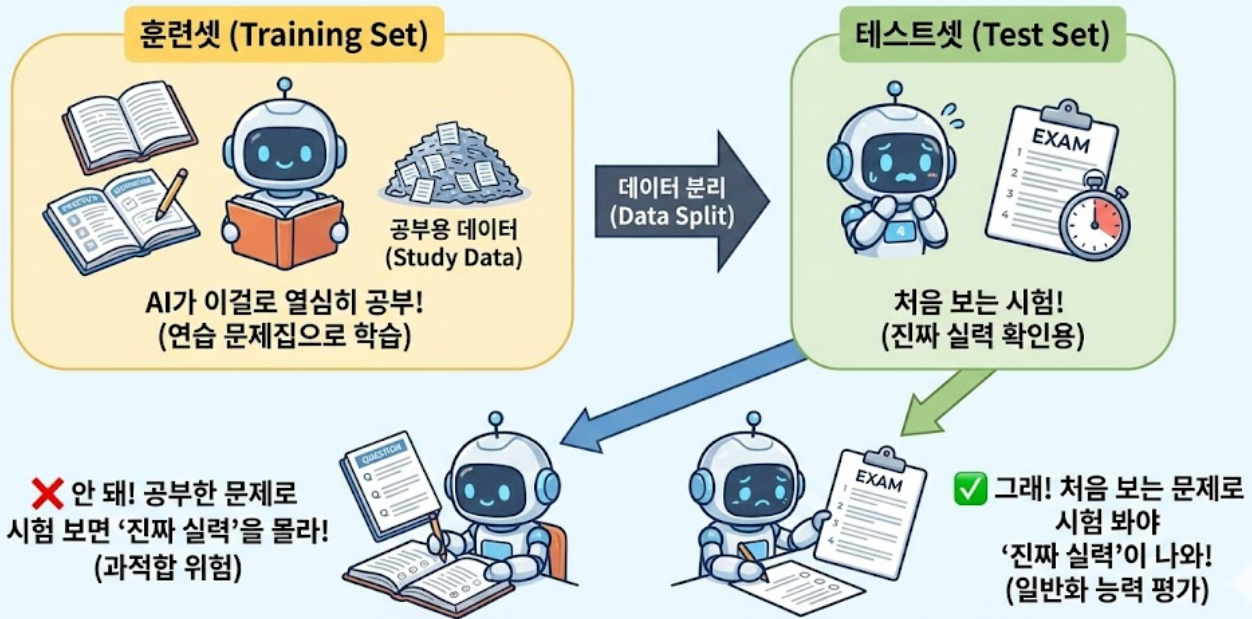
"이렇게 생각해봐요. 학생한테 수학 문제 20개를 줬어요. 그걸로 열심히 공부했죠. 그리고 시험 날, **그 20개 문제가 그대로** 나왔어요. 100점 받았어요. 그게 진짜 수학 실력이에요?"

"아... 아니죠. 그건 문제를 외운 거죠."

"맞아요! 지금 메타코딩 씨의 AI가 딱 그 상황이에요."

김선배가 냅킨에 그림을 그리기 시작했습니다.

훈련셋(Training Set) vs 테스트셋(Test Set) 나눠야 하는 이유: '진짜 실력' 검증!



"학습에 쓰는 데이터를 **훈련셋(Training Set)**, 평가에 쓰는 데이터를 **테스트셋(Test Set)** 이라고 해요. 이 둘은 반드시 분리해야 합니다. 시험 문제가 연습 문제랑 같으면 안 되잖아요."

📖 훈련셋 vs 테스트셋 — AI에서 가장 중요한 원칙

메타코딩은 이 개념을 노트에 크게 적었습니다.

[학교 시험에 비유]

연습 문제집 (훈련셋):

1번: $2 + 3 = ?$
2번: $5 \times 4 = ?$
...
20번: $8 - 2 = ?$

이걸로 공부

시험지 (테스트셋):

21번: $7 + 8 = ?$
22번: $3 \times 9 = ?$
...
30번: $6 \div 2 = ?$

처음 보는 문제!
이걸로 진짜 실력 평가!

—AI—
학습

이 원칙은 AI 분야 전체에서 통용됩니다. ChatGPT도, 이미지 인식 AI도, 넷플릭스 추천 AI도, 모두 **훈련셋**과 **테스트셋**을 분리합니다.

💡 어렵게 느껴지면 이것만 기억하세요!

- **훈련셋** = 공부할 때 쓰는 문제집 (이걸로 배움)
- **테스트셋** = 시험지 (처음 보는 문제! 이걸로 진짜 실력 평가)
- **핵심: 공부에 쓴 문제로 시험 보면 안 된다!** (그건 외운 것일 뿐)

🔧 v3의 수정

수정 자체는 놀랍도록 간단했습니다. 20개 맵을 섞어서 **16개는 학습용, 4개는 시험용**으로 나눴습니다.

```
# v3의 핵심 변경: 맵을 훈련용과 시험용으로 나눈다!

# 20개 맵을 섞어서 16개는 훈련용, 4개는 시험용으로 나누기
all_map_ids = list(range(20))          # [0, 1, 2, ..., 19]
random.shuffle(all_map_ids)            # 랜덤으로 섞기!
train_map_ids = all_map_ids[:16]        # 16개로 학습
test_map_ids = all_map_ids[16:]         # 4개로 시험 (처음 보는 맵!)

# 훈련할 때는 train_map_ids에 있는 맵만 사용!
# 테스트할 때는 test_map_ids에 있는 맵으로 진짜 실력 확인!
```

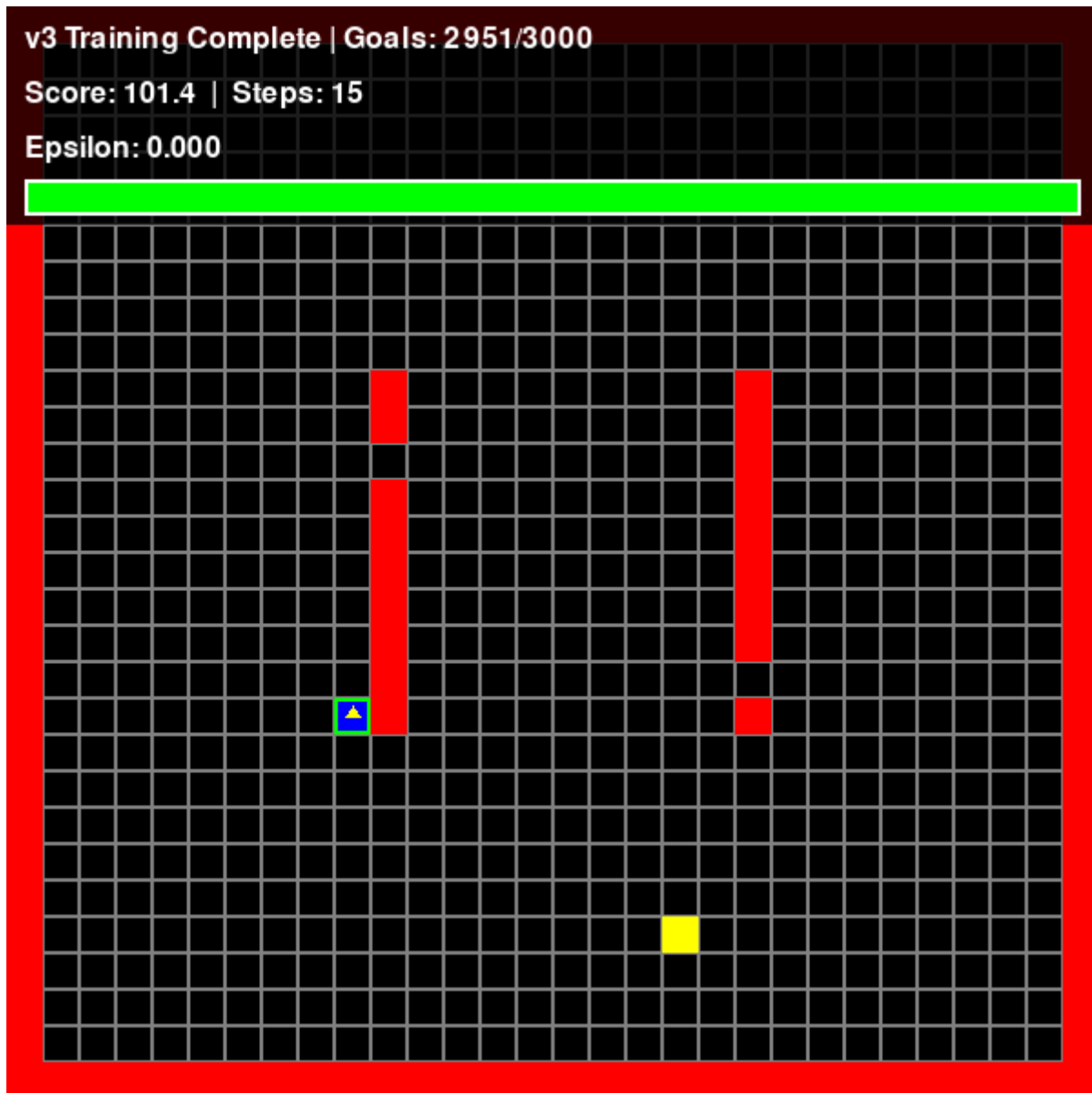
코드 몇 줄의 변경이지만, 그 의미는 거대했습니다. "거울 앞에서 연습하던 것"을 "진짜 무대에 세우는 것"으로 바꾼 것이니까요.

직접 실행해보기!

💡 지금 바로 v3를 실행해볼 수 있습니다!

학습 실행:

```
cd simulator-v3
python train.py
```

학습된 AI를 새로운 맵에서 테스트:

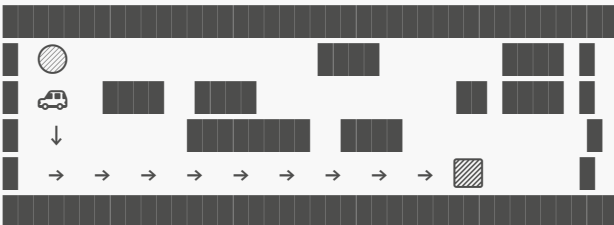
```
cd simulator-v3
python main.py
```



🎮 충격적인 결과

⚠️ 학습에 사용하지 않은 새로운 맵에서 테스트합니다!

[완전히 새로운 맵에서 v3 테스트]



Success Rate: 68% ← 진짜 실력!

v2에서 90%라고 믿었던 성공률이 사실은 **68%**였습니다.

"22%나 차이가 나네..."

메타코딩은 충격을 받았습니다. 하지만 동시에 중요한 것을 깨달았습니다. **모르고 있는 것보다, 진실을 아는 게 훨씬 낫다.**

v2에서 착각한 것:
"90% 성공! 우리 AI 대단하다!"

v3에서 발견한 진실:
"사실 68%. 22%는 맵을 외운 것."

하지만!
68%도 대단한 거예요!
v0은 복잡한 맵에서 0%였잖아요.
적어도 이제 '진짜 실력'을 측정할 수 있게 됐어요.

"성공률"의 진짜 이름: 정확도(Accuracy)

그런데 김선배가 한 가지 더 알려줬습니다.

"메타코딩 씨, 지금 쓰는 '성공률'이라는 말 있잖아요. AI 세계에서는 이것 ****정확도(Accuracy)****라고 불러요."

우리가 쓰는 "성공률" = AI 용어로 "정확도(Accuracy)"!

자율주행 예:
정확도 68% = 100번 중 68번 성공 = 우리가 쓰는 성공률!

이미지 분류 예:
정확도 95% = 고양이 사진 100장 중 95장을 맞힘!

→ 같은 개념, 다른 이름일 뿐!

AI 세계에서는 정확도 외에도 다양한 평가 지표가 있지만, 우리 자율주행에서는 ****정확도(성공률)****만으로 충분합니다!

과적합을 막는 다른 방법들

김선배가 노트에 한 가지 더 적어줬습니다.

"훈련셋/테스트셋 분리가 과적합을 **발견**하는 방법이라면, 과적합을 **막는** 방법도 있어요."

방법 1: L2 규제 (Weight Decay) — "한 명에게만 의존하지 마!"

L2 규제 = 가중치가 너무 커지면 패널티를 준다!

비유: 축구팀 감독이 선수를 기용하는 방식

나쁜 팀 (L2 규제 없음):

메시한테만 공을 몰아줌! 나머지 10명은 구경만...

→ 메시가 부상당하면? → 팀 전체가 무너짐! 💀

가중치: $[0.0, 0.0, 9.9]$

↑수비 무시 ↑미드 무시 ↑공격수 하나에 올인!

좋은 팀 (L2 규제 있음):

모든 선수가 골고루 역할을 나눠서 플레이!

→ 한 명이 빠져도 팀이 돌아감!

가중치: $[0.3, 0.2, 0.5]$

↑수비도 ↑미드도 ↑공격도 골고루 활용!

우리 AI에 적용하면?

규제 없음: "목적지 x거리" 가중치만 9.9로 폭주!

→ 목적지 방향만 보고 달림 → 벽에 부딪힘! ✖

규제 있음: 벽 정보, 방향, 거리를 골고루 참고

→ 벽도 피하고, 방향도 보고, 거리도 계산!

방법 2: L1 규제 (Lasso) — "쓸모없는 건 아예 0으로!"

L1 규제(라쏘)는 L2보다 더 과감한 방법입니다. 쓸모없는 가중치는 아예 0으로 만들어버립니다!

[라쏘(Lasso) 규제 – 쉬운 비유]

이사를 앞둔 상황이라고 생각해봅시다!

짐이 너무 많아서 트럭에 다 못 실어요.

"진짜 중요한 물건만 골라서 가져가자!"

L2 규제 (Weight Decay) 방식:

→ 모든 짐을 조금씩 줄인다

→ 큰 박스는 작은 박스로 교체

→ 모든 물건을 다 가져가되, 작게 만들

→ 결과: $[0.08, 0.15, 0.25]$ (다 조금씩 줄었지만 전부 남아있음)

L1 규제 (Lasso) 방식:

→ 쓸모없는 짐은 아예 버린다!

→ "이 낡은 우산? 버려!" "이 고장난 시계? 버려!"

→ 진짜 필요한 것만 남김

→ 결과: $[0.0, 0.0, 0.45]$ (쓸모없는 건 0, 중요한 건 남김!)

자율주행 예시:

11개 입력 중에서 "왼쪽아래 벽"이 별로 중요하지 않다면?

L1 규제: 그 가중치를 아예 0으로! → 더 깔끔하고 빠른 판단!

"우리 v3에서는 이 기법을 쓰지 않았지만, 나중에 모델이 더 복잡해지면 반드시 필요해져요."

정리하면, L2 = 골고루 줄이기, L1 = 쓸모없는 건 아예 0으로 버리기. 둘 다 목적은 같습니다: "AI가 외우지 말고 진짜 실력을 기르도록" 돕는 것!

📁 팀장님께 솔직하게 보고

다음날 메타코딩은 팀장님께 솔직하게 보고했습니다. "팀장님, v2의 90%는 정확하지 않았습니다. 진짜 새로운 환경에서 테스트하니 68%였습니다."

팀장님이 잠시 침묵했습니다. 메타코딩의 심장이 빠르게 뛰었습니다.

"메타코딩 씨, 이게 오히려 좋아요."

팀장님이 고개를 끄덕이며 말했습니다.

"68%가 진짜 실력이고, 이제 뭘 개선해야 하는지 방향이 생겼잖아요. 90%라고 착각하고 있었으면 어떻게 됐겠어요? 제품 출시하고 나서 엉망인 걸 발견했겠죠. 그게 더 큰 재앙이에요."

메타코딩은 고개를 끄덕였습니다. 정직한 측정이 있어야 정확한 개선이 가능하다.

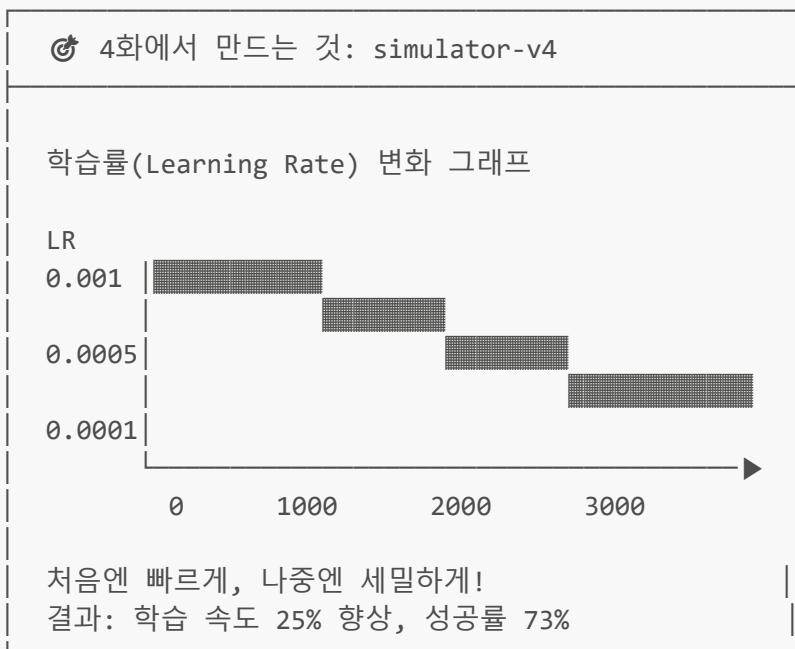
팀장님이 덧붙였습니다.

"그런데 학습이 3000번이나 걸리잖아요. 꽤 오래 걸리던데... 더 빨리 배우게 할 방법은 없을까요?"

메타코딩의 머릿속에 2화에서 배운 경사하강법이 스쳤습니다. "처음엔 큰 보폭, 나중엔 작은 보폭"이라는 전략.

"찾아보겠습니다!"

4화: 처음엔 크게, 나중엔 섬세하게 (v4)



📍 지금 배우는 것 → 최적화 기법 (딥러닝의 핵심)
지도학습 | 비지도학습 | 강화학습 ★ | 딥러닝 ★

📅 4주차 — 경사하강법의 실전 적용

v3를 완성하고 나서 메타코딩은 유튜브에서 강화학습 강의를 보다가 흥미로운 기법을 발견했습니다.

"Learning Rate Scheduling"

2화에서 경사하강법을 배울 때, "처음엔 큰 보폭으로 전체를 훑고, 나중엔 작은 보폭으로 정밀하게 찾는다"가 가장 효율적이라고 했죠. 그 전략을 실제 코드로 구현하는 것입니다!

🧠 Learning Rate Scheduling: 보폭 자동 조절

"처음엔 성큼성큼, 나중엔 살금살금!"

[비유: 새 교과서를 공부하는 방법]

1단계: 빠르게 훑어보기 (큰 Learning Rate = 0.001)

"우선 목차랑 소제목만 쪽 읽어보자!"

→ 전체 구조 파악!

→ "아, 이 책은 대충 이런 내용이구나!"

2단계: 본문 차근차근 읽기 (중간 Learning Rate = 0.0005)

"이제 각 챕터를 하나씩 읽어보자"

→ 핵심 개념 이해!

3단계: 중요 부분 정독 (작은 Learning Rate = 0.0001)

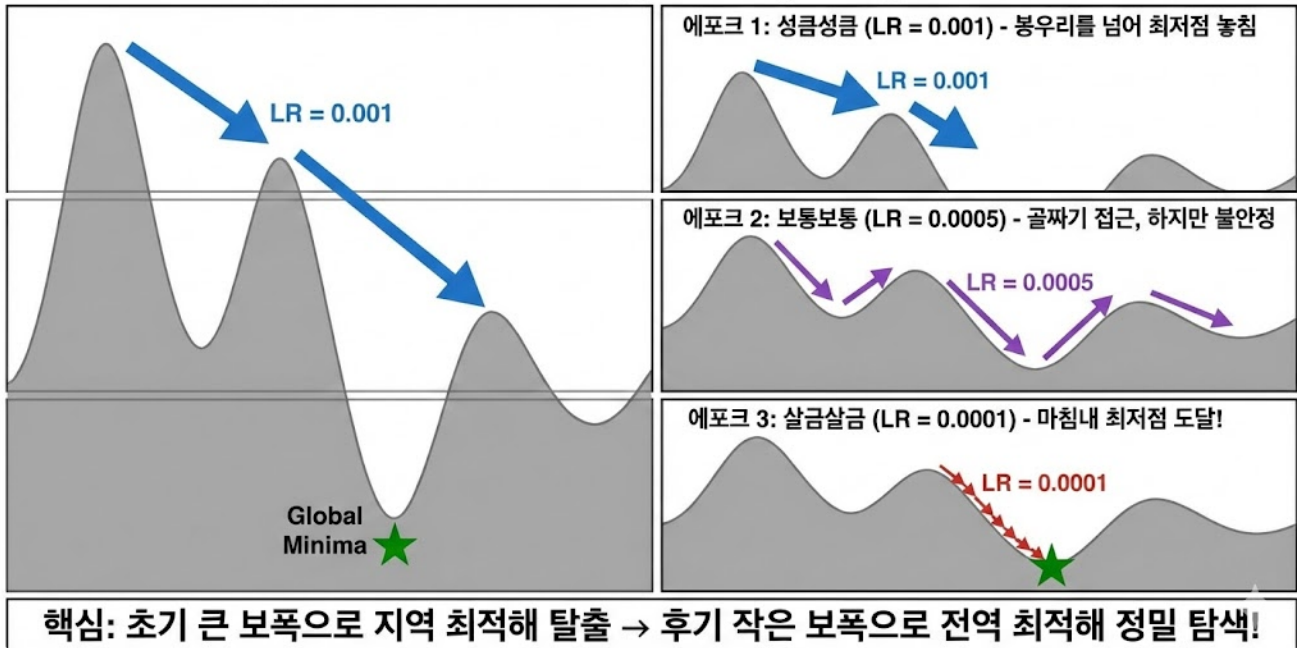
"이 부분은 핵심이니까 세 번 읽자!"

→ 디테일까지 완벽히!

→ 처음부터 한 글자씩 정독하는 것보다 훨씬 효율적!

경사하강법의 산 비유로 돌아가면, 처음에는 큰 보폭으로 전체를 훑고, 나중에는 작은 보폭으로 정밀하게 찾는 것이 핵심이었죠!

Learning Rate Scheduling: Navigating Local Minima



🔑 v4 코드 소개

```
class DQNAgent:
    def __init__(self):
        self.initial_lr = 0.001 # 처음: 큰 보폭으로 빠르게
        self.min_lr = 0.0001 # 마지막: 작은 보폭으로 세밀하게

    def update_learning_rate(self, episode):
        # 에피소드가 진행될수록 학습률이 줄어듦
        decay_rate = 0.998
        self.current_lr = self.initial_lr * (decay_rate ** episode)
        self.current_lr = max(self.current_lr, self.min_lr)

        # 옵티마이저에 적용
        for param_group in self.optimizer.param_groups:
            param_group['lr'] = self.current_lr
```

`decay_rate = 0.998`은 매 에피소드마다 학습률을 0.2%씩 줄인다는 뜻입니다. 아주 조금씩, 천천히 보폭을 줄여나가는 것이죠.

🤖 우리 AI가 쓰는 옵티마이저: Adam!

LR Scheduling 코드를 보다가 메타코딩이 또 다른 것을 발견했습니다.

"선배님, 코드에 `Adam`이라는 게 있는데 이건 뭐예요?"

김선배가 웃으며 설명했습니다.

"경사하강법을 기억하죠? 오차를 줄이는 방향으로 가중치를 수정하는 거. 그 경사하강법에도 종류가 있어요."

옵티마이저(Optimizer) = 경사하강법의 업그레이드 버전들!

SGD (기본 경사하강법):

모든 가중치를 같은 보폭으로 수정

비유: 직진만 하는 초보 스키어

→ 단순하지만, 울퉁불퉁한 지형에서 비효율적

Adam (우리가 쓰는 것!):

각 가중치마다 보폭을 자동으로 조절!

비유: 지형에 맞춰 자세를 바꾸는 프로 스키어

→ 가파른 곳은 조심, 완만한 곳은 빠르게!

산에서 최저점을 찾는 비유:

SGD: ●→→→→→→→→→→→→→→→●

(같은 보폭으로 쭉 직진!)

→ 때로 지그재그로 방향이 흔들림

Adam: ●→→→→→ →→ → → ●

(처음엔 크게, 나중엔 상황에 맞춰!)

→ 더 빠르고 안정적으로 도착!

"Adam은 'Adaptive Moment Estimation'의 줄임말인데, 이름은 몰라도 돼요. 중요한 건 **SGD보다 똑똑하게 가중치를 수정한다는 것!**"

우리 AI 코드에서는 이미 Adam을 사용하고 있습니다.

```
# agent.py에서 (v1부터 쭉 사용 중!)
self.optimizer = optim.Adam(self.model.parameters(), lr=0.001)
#           ^^^^^
#           SGD 대신 Adam을 쓰고 있었던 것!
```

💡 어렵게 느껴지면 이것만 기억하세요!

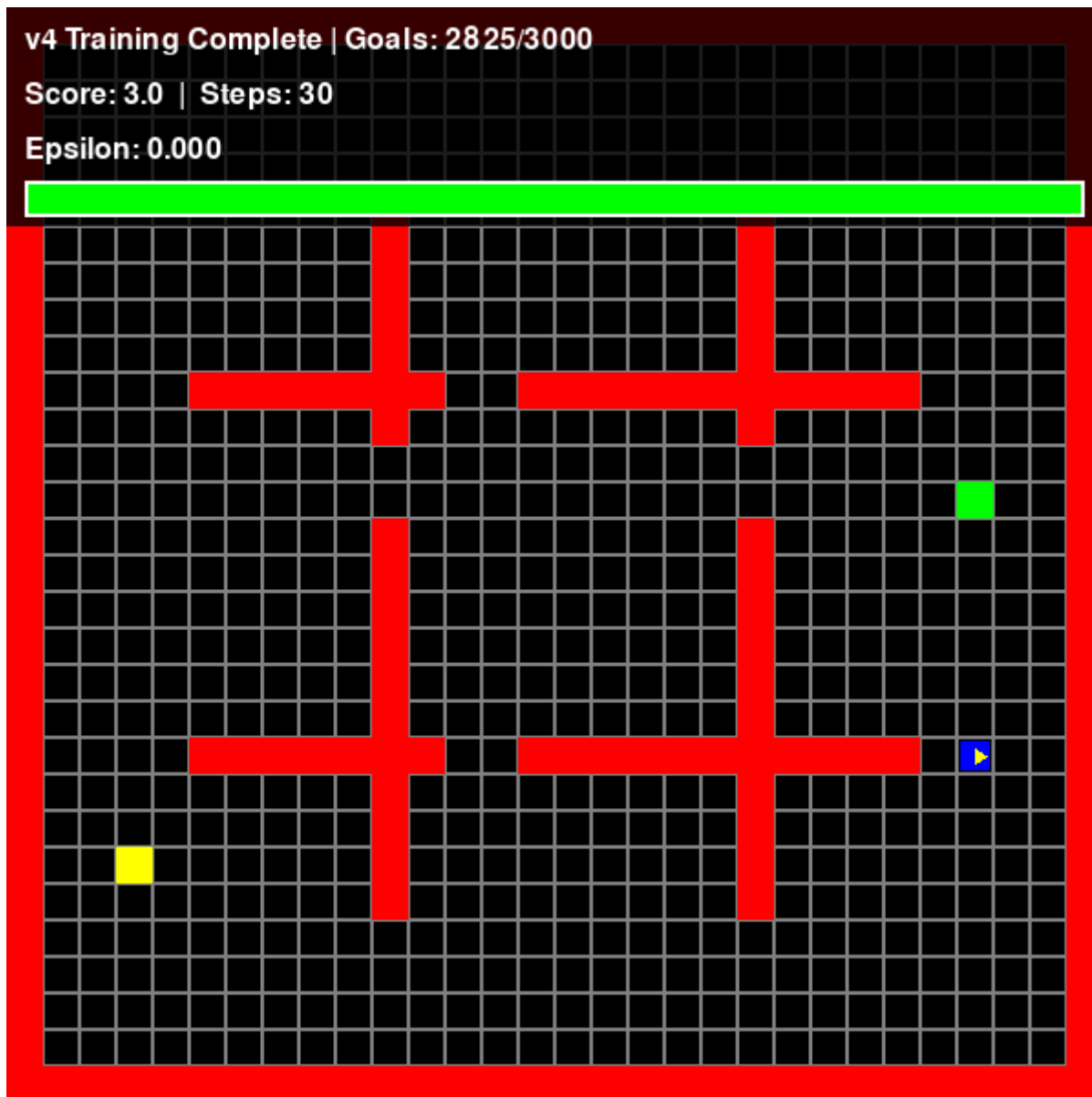
- **옵티마이저** = 경사하강법의 종류 (가중치를 어떻게 수정할지 결정)
- **SGD** = 기본형. 같은 보폭으로 직진 (초보 스키어)
- **Adam** = 업그레이드형. 상황에 맞춰 보폭 자동 조절 (프로 스키어)
- 실무에서는 대부분 **Adam**을 씁니다!

📖 직접 실행해보기!

💡 지금 바로 v4를 실행해볼 수 있습니다!

학습 실행 (Learning Rate가 자동으로 줄어드는 것을 관찰!):


```
cd simulator-v4  
python train.py
```



학습된 AI로 실행:

```
cd simulator-v4  
python main.py
```



🎮 v4 실행 결과

학습 진행:

Episode 100		Score: 1.2		LR: 0.000820	←	아직 큰 보폭
Episode 500		Score: 3.8		LR: 0.000368	←	점점 줄어듦
Episode 1000		Score: 5.2		LR: 0.000135	←	중간 보폭
Episode 2000		Score: 6.1		LR: 0.000100	←	세밀한 보폭
Episode 3000		Score: 6.8		LR: 0.000100	←	최소값 유지

[v4 테스트 - 새로운 맵에서]

A diagram of a new 2D map layout for testing. It shows a rectangular area with a black background and a red border. The map contains several black rectangular obstacles of varying sizes. A yellow circle with a black outline is located in the top-left corner, and a yellow car icon is positioned below it. A series of arrows indicates a path starting from the car, moving down, then right, and finally down again to reach a goal area on the right side of the map.



"맞아요. 이미 아는 집에서는 더 빠르게 움직일 방법이 없을까요?"

55 / 81

"선배님, 팀장님이 추론 속도를 최적화하라고 하시는데... 추론이 뭔가요?"

김선배가 웃으며 화이트보드에 적었습니다.

AI의 두 단계:

학습 (Training):

- AI가 "배우는" 과정
- 공장에서 한 번 (몇 시간~며칠)
- 끝나면 모델 파일(.pth)로 저장
- 느려도 괜찮음!

추론 (Inference):

- 배운 걸 "실제로 사용하는" 과정
- 매번 청소할 때마다!
- 빠를수록 좋음! (배터리 절약, 반응 빠름)

사람으로 비유하면:

학습 = 운전면허 학원 다니기 (한 번)

- 몇 주 걸려도 괜찮음
- 열심히 배우면 됨

추론 = 매일 아침 출근 운전 (매일)

- 빠르고 자연스러워야 함
- 매번 교과서 펴보면서 운전할 수 없잖아요!

"학습은 한 번이면 되지만, 추론은 청소할 때마다 해야 하잖아요. 매번 신경망 45ms를 기다리는 건 비효율적이네요."

💡 메타코딩의 아이디어: 캐싱!

"그리고 보니... 같은 집에서 매일 청소하면 비슷한 경로로 다니는데, 매번 신경망 계산을 하는 게 낭비 아닌가? 이미 성공한 경로를 저장해두면 더 빠르지 않을까?"

네비게이션에 비유하면:

✗ 매번 새로 계산:

"서울→부산 경로를 다시 계산합니다... 45ms"
(매일 같은 길인데 매번 계산?)

☑ 자주 가는 길은 저장:

"저장된 경로입니다! 바로 안내합니다. 12ms"
(3.75배 빠름!)

🔑 캐싱 시스템 코드 소개

```
class ActionCache:
    """성공한 경험을 저장하는 캐시"""
    def __init__(self):
        self.cache = {} # 상태 → 행동 매핑

    def get(self, state):
        """이 상황을 본 적 있나?"""
        state_key = tuple(round(x, 2) for x in state)
        return self.cache.get(state_key, {}).get('action', None)

    def store(self, state, action, success):
        """성공한 행동만 저장!"""
        if success:
            state_key = tuple(round(x, 2) for x in state)
            self.cache[state_key] = {'action': action}
```

핵심은 **성공한 경험만 저장**한다는 것입니다. 실패한 경험까지 저장하면 "여기서 벽에 부딪혔었지"라는 나쁜 기억을 따라갈 수도 있으니까요.

💡 두 가지 전략: 정적 vs 동적 환경

김선배가 중요한 포인트를 짚어줬습니다.

"캐시를 쓸 때는 환경에 따라 전략을 달리해야 해요."

전략 1: 정적 환경 (공장, 창고 — 장애물이 고정)

캐시에 있으면 → 캐시 사용! (12ms, 3.75배 빠름!)

캐시에 없으면 → 신경망 계산 (45ms)

매일 같은 구조인 창고에서는 캐시가 매우 효과적!

전략 2: 동적 환경 (가정집 — 가구 이동, 사람 이동)

항상 신경망(Policy)이 먼저! (45ms, 하지만 안전!)

캐시는 참고만 → 캐시와 다르면 신경망 우선!

사람이 돌아다니는 집에서는 안전이 최우선!

운전자(Policy)가 네비(Cache)보다 우선인 이유:

네비: "이 길로 직진하세요"

운전자: "어? 앞에 공사중이야! 우회해야겠어!"

→ 실시간 판단(Policy)이 저장된 기억(Cache)보다 중요!

→ 눈앞의 현실이 과거의 기억보다 정확!

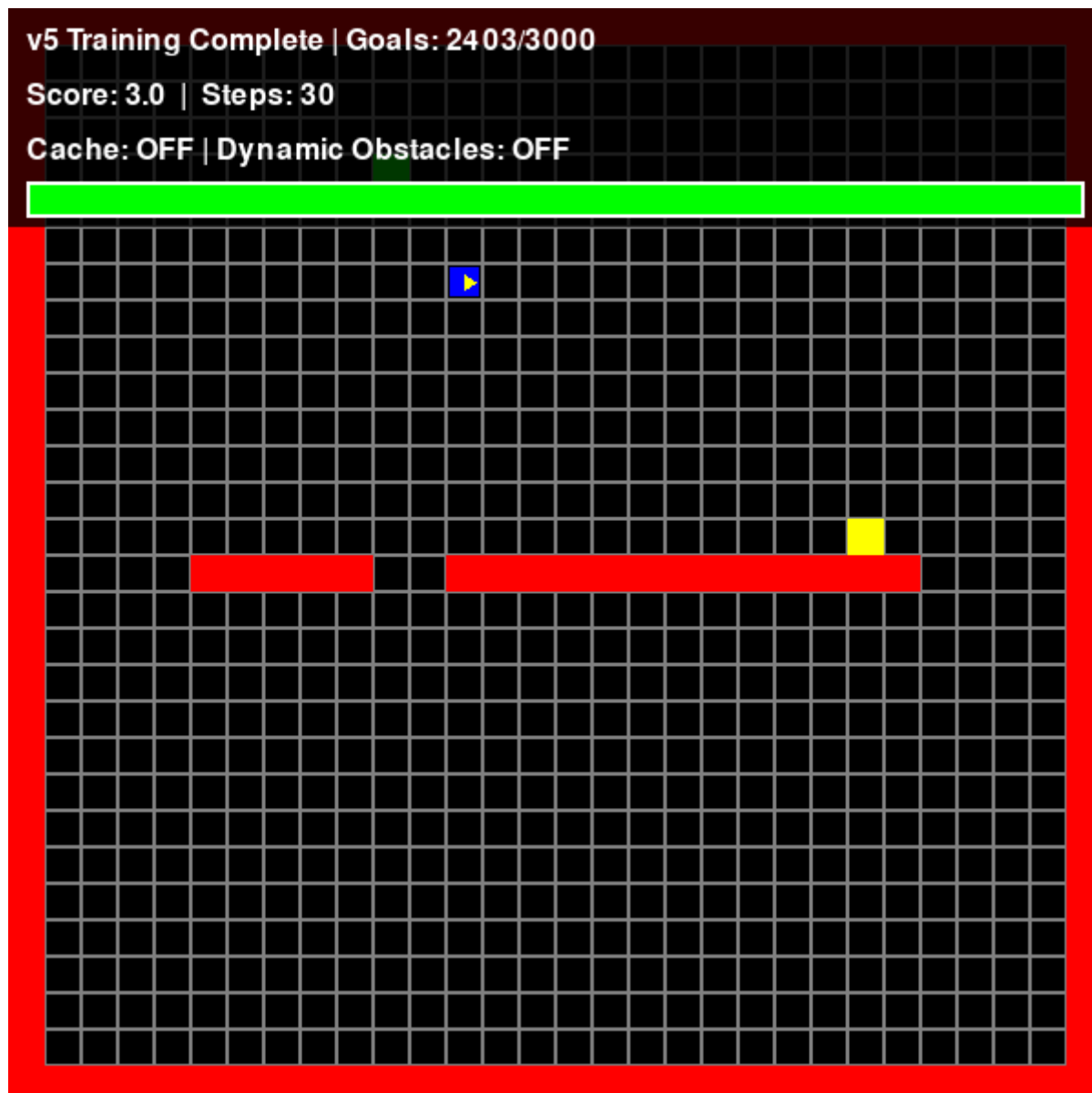
한 줄로 정리하면: 캐시 = 네비 즐겨찾기, Policy = 운전자의 눈. 저장된 길보다 눈앞의 현실이 더 중요합니다!

🖥️ 직접 실행해보기!

💡 지금 바로 v5를 실행해볼 수 있습니다!

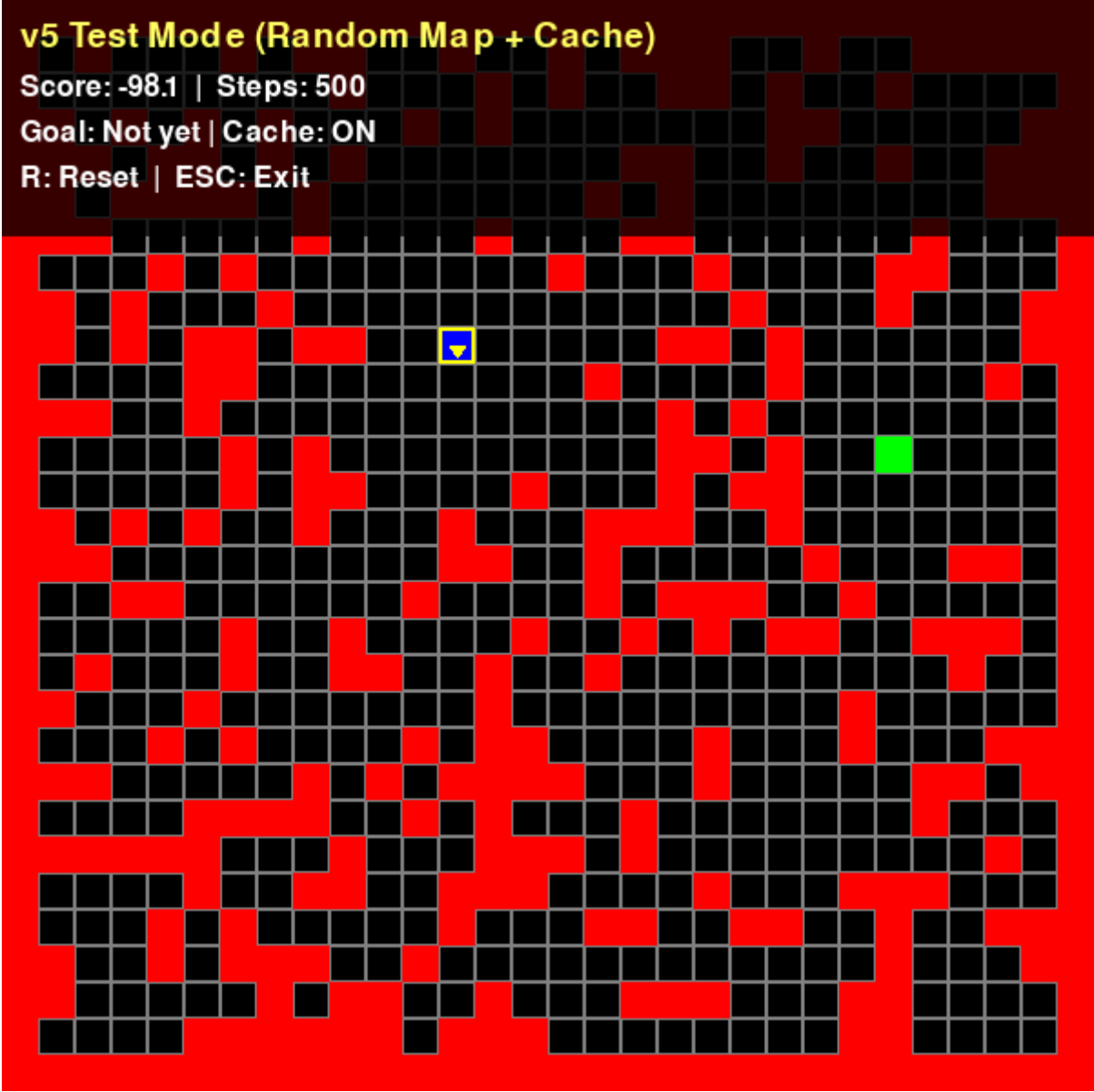
학습 실행:

```
cd simulator-v5
python train.py
```



학습된 AI + 캐싱 시스템으로 실행:

```
cd simulator-v5
python main.py
```



💡 캐시 히트(Cache Hit)가 뜨는 것을 관찰해보세요! 캐시를 사용할 때 얼마나 빨라지는지 확인할 수 있습니다.

🎮 v5 실행 결과

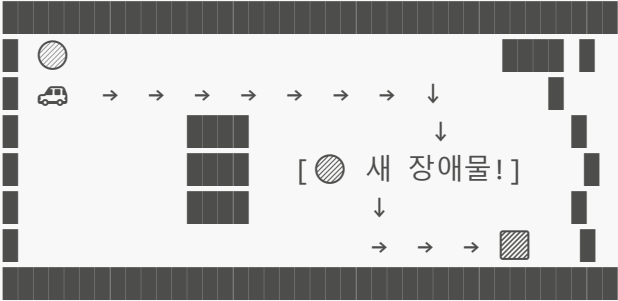
정적 환경 (장애물 고정):

[v5 - 정적 환경]

Cache: Size=350 | Hits=142 | Agreements=89 | Conflicts=8
☑ Success Rate: 73%
캐시 덕분에 추론 속도 3.75배 향상!

동적 환경 (50스텝 후 새 장애물 등장!):

[v5 - 동적 환경, Step 51]



⚠ 캐시: "직진" vs Policy: "장애물 감지! 우회!"
→ Policy 우선으로 안전하게 우회! ☒

Success Rate: 65% (캐시만 믿었다면: 30%)

📁 최종 보고

팀장님: "좋아요! 정적 환경에서는 빠르고, 동적 환경에서도 안전하네요. 이제 제품화 고민을 시작할 수 있겠어요!"

5주간의 여정... 드디어 제품화 가능 수준에 도달했습니다.

최종 성능 정리:

새로운 맵 성공률	: 73%
학습 속도 개선	: v3 대비 25% 향상 (v4)
추론 속도 개선	: 정적 환경에서 3.75배 빠름 (v5)

하지만 73%는 아직 부족합니다. 팀장님의 목표는 **90%** 이니까요.

메타코딩은 고민에 빠졌습니다. "DQN을 더 개선해야 하나? 아니면 완전히 다른 접근이 필요한 건가?"

그때 김선배가 한 마디를 던졌습니다.

"메타코딩 씨, **DQN만이 정답은 아니에요**. 같은 문제를 완전히 다른 방식으로 풀 수도 있어요."

번외편: 망치 너머의 도구들 (simulator-v6)

번외편: 망치 너머의 도구들 (simulator-v6)

1단계: v5 결과로 데이터 만들기
2단계: 의사결정 트리 - "질문"으로 판단하는 AI
3단계: 랜덤 포레스트 - 100개 트리의 "투표"
4단계: 트리/포레스트 AI로 자율주행!
5단계: DQN vs 트리 vs 포레스트 비교!
+ RNN, LSTM, 트랜스포머 개념 소개

♀ 지금 배우는 것 → 지도학습
 지도학습 ★ | 비지도학습 | 강화학습 | 딥러닝
 (DQN 말고 다른 방법으로도 자율주행을 만들어보자!)

다른 방법이 있다고?

v5까지의 여정이 끝난 어느 날이었습니다. 메타코딩은 퇴근 준비를 하다가 김선배를 마주쳤습니다.

"메타코딩 씨, 잘했어요. 73%까지 올리다니."

김선배가 커피잔을 들며 말했습니다.

"감사합니다. 그런데 90%까지는 아직 갈 길이 멀어요."

메타코딩이 한숨을 쉬었습니다. 5주 동안 v0부터 v5까지, 규칙 기반에서 DQN까지 달려왔는데 아직도 27%가 남아 있다니.

김선배가 갑자기 물었습니다.

"그런데 **DQN** 말고 다른 방법으로도 자율주행을 만들 수 있는 거 알아요?"

"네? 다른 방법이요?"

"DQN은 신경망으로 복잡한 계산을 하잖아요. 근데 '**질문**'만으로 판단하는 AI도 있어요. 훨씬 간단하고, 왜 그렇게 판단했는지 사람이 이해할 수도 있어요."

메타코딩의 눈이 동그아졌습니다.

"질문만으로요? 그게 가능해요?"

김선배가 웃으며 노트를 꺼냈습니다.

"**의사결정 트리**라는 거예요. 그리고 그 트리를 100개 모으면 **랜덤 포레스트**가 되고요."

메타코딩은 노트에 적기 시작했습니다.

"DQN만이 정답은 아니다. 같은 문제를 다른 각도에서 풀어보자!"

오늘의 여정: 5단계로 따라하기!

이번 번외편은 **5단계**로 진행됩니다. 각 단계마다 먼저 "이게 뭔지" 한 줄로 알려드리고, 바로 실행해서 결과를 본 다음, 자세하게 설명합니다.

오늘의 여정:

```
cd simulator-v6
```

1단계: python collect_data.py	← v5 결과로 데이터 만들기
2단계: python train_tree.py	← 의사결정 트리 학습
3단계: python train_forest.py	← 랜덤 포레스트 학습
4단계: python run_tree.py	← 트리/포레스트 AI로 자율주행!
5단계: python compare.py	← DQN vs 트리 vs 포레스트 비교!

💡 **v5 학습된 모델이 필요합니다!** 아직 v5를 실행 안 했다면, 먼저 `simulator-v5`에서 `python train.py`로 학습을 완료해주세요.

자, 그럼 1단계부터 시작해볼까요?

1단계: v5 결과로 데이터 만들기 (collect_data.py)

한 줄 요약: v5에서 학습된 DQN이 운전하는 모습을 녹화해서, 다른 AI의 교과서를 만든다!

실행해보기!

```
cd simulator-v6
python collect_data.py
```

결과 확인

```
에피소드 10/100 | 성공: 7 | 수집된 데이터: 245개
에피소드 20/100 | 성공: 15 | 수집된 데이터: 512개
...
에피소드 100/100 | 성공: 73 | 수집된 데이터: 2847개
=====
데이터 수집 완료!
성공 에피소드: 73/100
총 데이터: 2847개
저장 위치: driving_data.csv
=====
```

`driving_data.csv` 파일이 생겼습니다! 이게 뭘까요?

이게 무슨 데이터인가요?

의사결정 트리와 랜덤 포레스트는 **지도학습** 방식입니다. 프롤로그에서 배운 것, 기억나시나요?

AI가 배우는 3가지 방식 (프롤로그 복습!):

1. 지도학습: "정답을 알려주고 배우기"
→ 선생님이 문제와 정답을 다 알려줌
2. 비지도학습: "정답 없이 패턴 찾기"
→ 선생님 없이 혼자 패턴을 발견
3. 강화학습: "시행착오로 배우기"
→ 보상과 패널티로 스스로 학습 ← DQN이 이거였죠!

지도학습은 "문제와 정답이 세트로 있어야" 합니다. 그런데 자율주행에서 정답을 어디서 구하죠?

바로 **v5의 DQN**에서 가져옵니다!

방금 collect_data.py가 한 일:

v5 DQN (선생님)

학습된 AI가
자율주행을 하면서
매 순간의 판단을
기록한다!

—저장—▶

driving_data.csv (답안지)

상태 → 행동
[0,1,0,0,0,1,0,0,1,
+0.6,+0.4] → 직진
[1,0,0,0,1,0,0,0,2,
-0.3,+0.2] → 좌회전
...약 2800개!

한 마디로: DQN 선생님이 푼 답안지를 보고, 트리 학생이 따라 배우는 것!

그리고 **성공한 에피소드의 데이터만** 저장합니다. 실패한 운전은 나쁜 습관이니까, 좋은 데이터만 교과서에 넣는 거예요!

```
# collect_data.py 핵심 부분

# 성공한 에피소드의 데이터만 저장! (좋은 데이터만 학습하도록)
if env.is_goal(car.x, car.y):
    for row in episode_data:
        writer.writerow(row)
```

교과서(데이터)가 준비됐으니 2단계로 넘어갑시다!

2단계: 의사결정 트리 학습 (train_tree.py)

한 줄 요약: "스무고개 게임"처럼 질문을 던져서 답을 찾는 AI!

실행해보기!

```
python train_tree.py
```

결과 확인

```
=====
의사결정 트리로 자율주행 배우기!
=====
학습 데이터: 2847개
상태 크기: 11개 (벽 8개 + 방향 + 목적지 x,y)
행동 종류: 3개 (0=직진, 1=우회전, 2=좌회전)

훈련 데이터 정확도: 92.3%
테스트 데이터 정확도: 85.7%

트리가 가장 중요하게 보는 정보:
1. 목적지Y: 0.312 ★★★★★
2. 목적지X: 0.278 ★★★★★
3. 위 벽: 0.145 ★★★
...
=====
```

오! 몇 초 만에 학습이 끝났고, 정확도가 85.7%나 나왔습니다! DQN은 수십 분 걸렸는데, 이건 눈 깜짝할 사이네요.

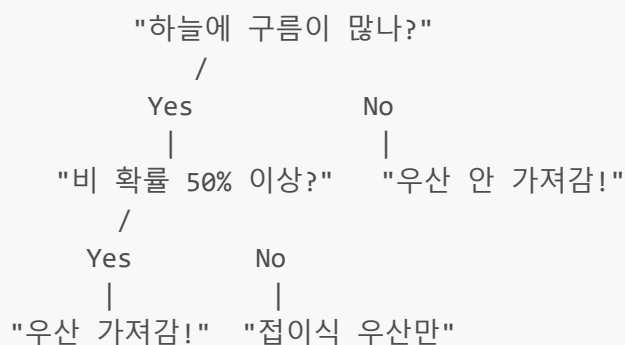
그런데 *****의사결정 트리*****가 정확히 뭘까요? 결과를 봤으니 이제 자세히 알아보시다!

의사결정 트리란?

의사결정 트리는 이름 그대로, **나무 모양의 질문**을 던져서 답을 찾아가는 AI입니다.

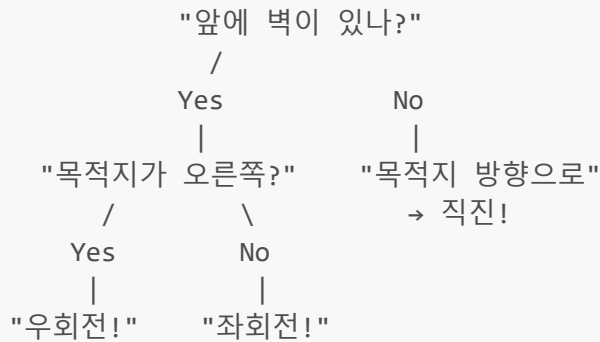
우리가 일상에서 하는 판단도 사실 이런 구조입니다. "오늘 우산을 가져갈까?"를 생각해 보세요.

[일상 속 의사결정 트리]

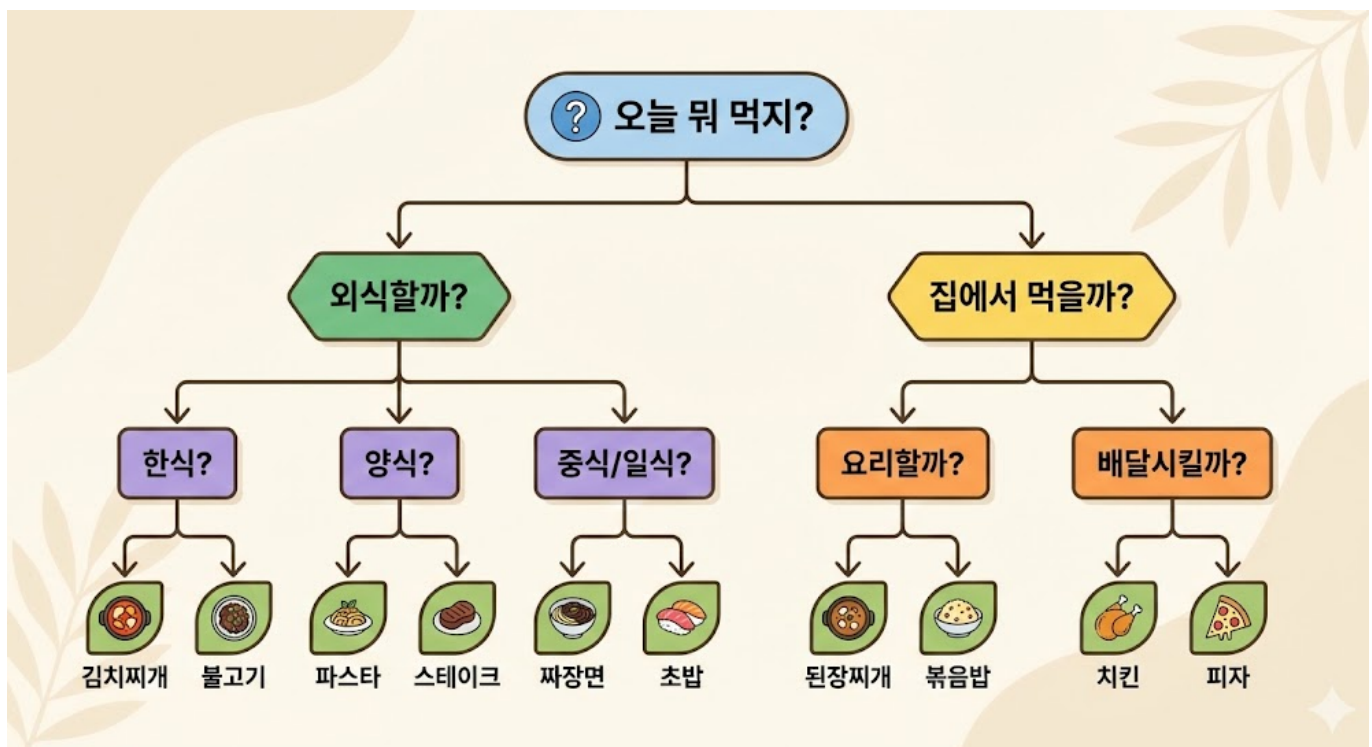


자율주행 AI도 똑같습니다!

[자율주행 의사결정 트리]



DQN은 숫자 수천 개를 곱하고 더하는 복잡한 계산으로 판단하지만, 의사결정 트리는 **질문과 대답만으로** 판단합니다. 마치 스무고개 게임처럼요!



트리가 자율주행을 배우는 과정

방금 실행한 `train_tree.py`에서 트리는 수집된 데이터를 보면서 스스로 질문을 만들어 갑니다.

```

# train_tree.py 핵심 부분

# 의사결정 트리 학습!
tree = DecisionTreeClassifier(max_depth=10, random_state=42)
tree.fit(X_train, y_train) # 데이터를 보면서 질문을 만든다!
  
```

`max_depth=10`은 "질문을 10번까지만 해라"라는 뜻입니다. 질문을 너무 많이 하면 과적합에 빠지거든요. v3에서 배운 것처럼요!

결과 다시 보기: 트리가 중요하게 보는 정보

아까 결과에 "트리가 가장 중요하게 보는 정보"가 나왔죠? 이것이 바로 **Feature Importance(특성 중요도)** 입니다.

트리가 가장 중요하게 보는 정보:

1. 목적지Y 방향 : 0.312 ★★★★★
2. 목적지X 방향 : 0.278 ★★★★★
3. 위 벽 : 0.145 ★★★
4. 아래 벽 : 0.089 ★★
5. 현재 방향 : 0.076 ★

→ "목적지가 어디인지"가 가장 중요하고,
"바로 앞에 벽이 있는지"가 그 다음으로 중요하다!

이게 바로 의사결정 트리의 최대 장점입니다. **"왜 그렇게 판단했는지 사람이 이해할 수 있다!"** 이걸 AI에서는 *****해석 가능성(Interpretability)*****이라고 부릅니다.

DQN은 어떤가요? 신경망 안에 숫자가 수천 개 들어있는데, 왜 직진을 선택했는지 사람이 들여다봐도 도무지 이해할 수 없습니다. 그냥 "컴퓨터가 계산했대"가 전부예요.

DQN (블랙박스):

입력 → [????복잡한 계산????] → "직진!"

사람: "왜 직진이야?"

DQN: ".....(수천 개 숫자를 보여줌)"

사람: "무슨 소리야?"

의사결정 트리 (해석 가능!):

"앞에 벽? No → 목적지가 오른쪽? No → 직진!"

사람: "아, 앞에 벽이 없고 목적지가 앞쪽이니까 직진이구나!"

사람: "이해가 돼!"

💡 어렵게 느껴지면 이것만 기억하세요!

- 의사결정 트리 = "스무고개 게임" 하듯이 질문을 던져서 답을 찾는 AI
- 최대 장점 = 왜 그렇게 판단했는지 사람이 이해할 수 있음! (해석 가능성)
- 최대 단점 = 복잡한 상황에서 **DQN보다 성능이 낮음**

3단계: 랜덤 포레스트 학습 (train_forest.py)

한 줄 요약: 트리 1개는 틀릴 수 있지만, 100개가 투표하면 정확해진다!

실행해보기!

```
python train_forest.py
```

결과 확인

```
=====
랜덤 포레스트로 자율주행 배우기!
=====
트리 100개 학습 중...

양상블 효과 비교:
트리 1개 정확도:          약 85%
트리 100개(포레스트) 정확도: 약 89%
→ 포레스트가 약 4% 더 정확! (투표의 힘!)
=====
```

메타코딩은 눈을 반짝였습니다. "트리 1개로 85%였는데, 100개를 합치니 89%로 올랐어요! 도대체 어떻게요?"

랜덤 포레스트란?

혼자 판단하면 틀릴 수 있지만, **100명이 투표하면** 정답에 가까워집니다. 이걸 AI에서는 ****양상블(Ensemble)****이라고 부릅니다.

천재 한 명에게 의존하는 대신, '조금씩 다르게 배운 평범한 전문가 100명'을 모아서 다수결로 결정하면 어떨까요? 이것이 바로 ****랜덤 포레스트(Random Forest)****의 핵심 아이디어입니다. 말 그대로 '나무(Tree)가 모여서 숲(Forest)을 이룬 것'이죠.

💡 **핵심은 '랜덤(Random)'에 있다!** 그냥 똑같은 트리 100개를 만들면 의미가 없습니다. 같은 사람 100명에게 물어보는 것과 같으니까요. 그래서 각 트리를 만들 때 **다른 데이터, 다른 질문**을 씁니다.

- **데이터를 랜덤하게 나눠준다 (Bagging):** 트리 1호는 전체 데이터 중 랜덤한 70%를 뽑아서 학습하고, 트리 2호는 또 다른 70%를 뽑아서 학습합니다. 각자 보는 데이터가 조금씩 다릅니다.
- **질문도 랜덤하게 제한한다 (Feature Randomness):** 어떤 트리는 '앞까지의 거리'만, 어떤 트리는 '목적지 방향'만 보도록, 판단 기준을 랜덤하게 제한합니다.

이렇게 하면 서로 조금씩 다른 시각을 가진 독립적인 트리 군단이 탄생합니다. 100개가 각자 다른 관점에서 판단하고, 투표하면 편향된 판단이 서로 상쇄됩니다!

[양상블의 원리 - 투표!]

```
트리 1호: "직진!"
트리 2호: "직진!"
트리 3호: "우회전!"
트리 4호: "직진!"
트리 5호: "직진!"
...
트리 99호: "직진!"
트리 100호: "직진!"
```

투표 결과: 직진 80표 / 우회전 15표 / 좌회전 5표
→ "직진!"

비유하면, 시험 범위가 100페이지인데 학생 A는 1~70페이지, 학생 B는 20~90페이지, 학생 C는 30~100페이지를 공부했어요. 혼자서는 모르는 범위가 있지만, 셋이 모여서 답을 맞춰보면 거의 모든 범위를 커버할 수 있죠!

코드로 보면

```
# train_forest.py 핵심 부분

# 랜덤 포레스트 학습! (트리 100개!)
forest = RandomForestClassifier(
    n_estimators=100,    # 트리 100개!
    max_depth=10,       # 각 트리는 질문 10번까지
    random_state=42
)
forest.fit(X_train, y_train)
```

`n_estimators=100`이 "트리 100개를 만들어라"라는 뜻입니다. 이 한 줄이 앙상블의 핵심이에요!

💡 어렵게 느껴지면 이것만 기억하세요!

- 랜덤 포레스트 = 트리 100개가 투표해서 결정!
- 왜 더 좋은가? = 한 명이 틀려도 다수가 보정해줌
- 핵심: 다양한 관점의 조합 > 한 명의 천재



4단계: 트리/포레스트 AI로 자율주행! (run_tree.py)

학습이 끝났으니 이제 직접 달려봅시다!

실행해보기!

```
python run_tree.py
```

시뮬레이터 창이 열리면서, 의사결정 트리와 랜덤 포레스트가 자율주행을 합니다! DQN과는 또 다른 느낌으로 움직이는 걸 볼 수 있어요.

5단계: DQN vs 트리 vs 포레스트 비교! (compare.py)

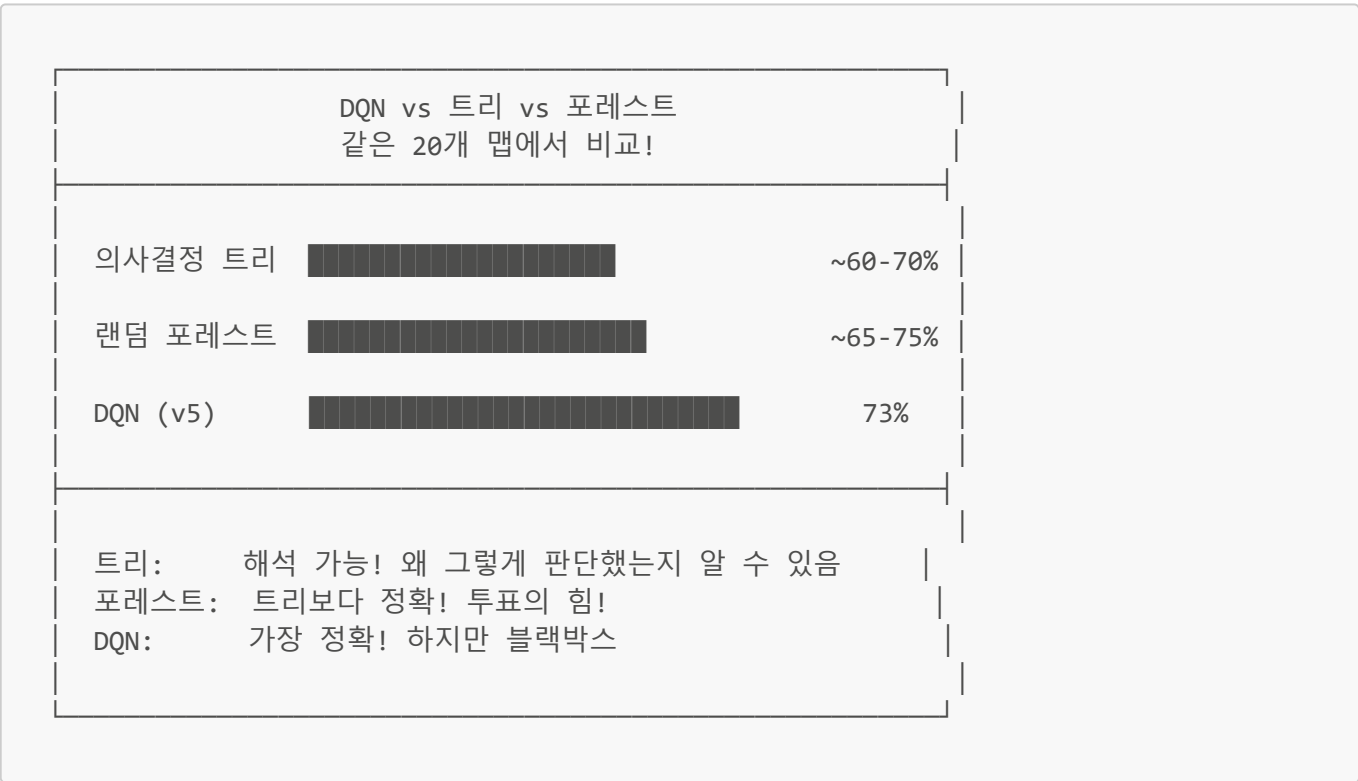
이제 궁금한 질문이 하나 남았습니다. "그래서 셋 중에 뭐가 제일 잘해요?"

실행해보기!

```
python compare.py
```

같은 맵에서 세 가지 AI가 자율주행을 시도하고, 성공률을 비교합니다! `comparison_result.png` 그래프도 저장됩니다.

결과 확인



숫자만 보면 DQN이 이기지만, 각각 장단점이 다릅니다.

세 AI의 장단점 비교:

	의사결정 트리	랜덤 포레스트	DQN
성공률	~60-70%	~65-75%	73%
학습 시간	매우 빠름	빠름	매우 느림
해석 가능성	★★★	★★	★
학습 방식	지도학습	지도학습	강화학습
복잡한 상황	약함	보통	강함
코드 복잡도	매우 간단	간단	복잡

김선배가 말했습니다.

"결국 은총알(**Silver Bullet**)은 없어요. 모든 상황에 완벽한 AI는 존재하지 않아요. 상황에 따라 적합한 도구를 고르는 것이 중요합니다."

언제 어떤 AI를 쓸까?

🌳 의사결정 트리를 쓰는 경우:

- "왜 이렇게 판단했어?"를 설명해야 할 때

- 의료 진단, 대출 심사 등 (이유를 알아야 하는 분야)

- 데이터가 많지 않을 때

🌲 랜덤 포레스트를 쓰는 경우:

- 트리보다 정확해야 하지만, 여전히 빠르게 학습하고 싶을 때

- 캐글(Kaggle) 데이터 분석 대회에서 자주 사용!

- 실무에서 "일단 빠르게 좋은 성능"이 필요할 때

🧠 DQN을 쓰는 경우:

- 복잡한 환경에서 최고 성능이 필요할 때

- 정답이 없고, 시행착오로만 배울 수 있을 때

- 게임, 로봇, 자율주행 등

한 마디로: "망치밖에 없으면 모든 게 못으로 보인다." AI도 도구함에 여러 도구가 있으면 좋습니다!

RNN, LSTM, 트랜스포머 개념 소개: 기억하는 AI

이 섹션은 개념만 소개합니다. 실제 코드는 없습니다!

메타코딩이 시뮬레이터를 보다가 말했습니다.

"김선배님, AI한테 긴 문장을 읽히면 중간 내용을 까먹지 않나요?"

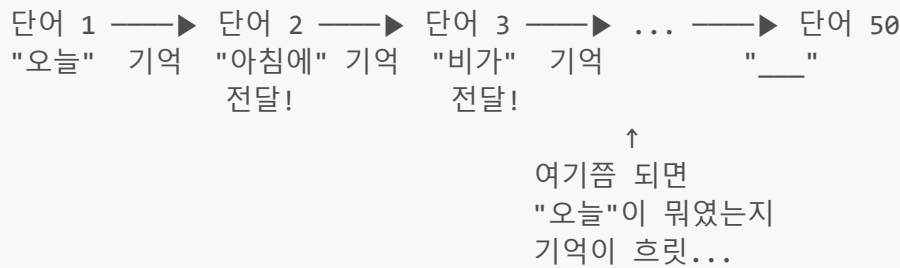
김선배가 고개를 끄덕였습니다.

"맞아요! 예를 들어 '오늘 아침에 비가 와서 우산을 챙겼는데, 점심에는 맑아지고, 저녁에는 다시 흐려져서 결국 __를 잘 챙겨온 셈이다' 같은 긴 문장에서, 앞부분의 '우산'을 뒤에서 기억해야 하잖아요. 이런 **긴 맥락을 기억하는 것**이 AI에게 어려운 문제예요."

RNN vs LSTM vs 트랜스포머 비교

1. RNN (순환 신경망) — 과거를 기억하지만 금방 잊음

RNN:



예시: "오늘 아침에 비가 와서 우산을 챙겼는데... (중간 30단어)... 결국 ___를 잘 챙겨왔다"

→ RNN: "뭘 챙겼다고? 앞에 뭐라고 했지...?" (중간에 까먹음!)

문제: 기억이 전화 게임처럼 점점 흐려짐!

1단어 전 ★★★★★ (선명!)

5단어 전 ★★★ (흐릿...)

20단어 전 ☆ (사라짐!)

2. LSTM (장단기 기억) — 중요한 건 오래 기억!

LSTM:

단어를 읽으면서 "이건 중요! 기억하자!" / "이건 사소, 버리자!"를 선택!

예시: 같은 긴 문장을 읽을 때

→ "우산"이 나왔을 때: "이건 핵심 키워드! 오래 기억하자!" ★★★★★

→ "맑아지고"가 나왔을 때: "별로 중요하지 않네, 잊어도 돼" ☆

→ 30단어 후: "아까 '우산'이 있었지? → '우산'을 잘 챙겨왔다!" ☑

RNN보다 훨씬 나음!

하지만... 문장이 수백 단어, 수천 단어가 되면?

→ 그래도 점점 기억이 약해짐 🙄

3. 트랜스포머 (Transformer) — 최근 LLM(ChatGPT 등)이 사용하는 방식!

트랜스포머의 핵심: "Attention(주의 집중)"

RNN/LSTM: 단어를 순서대로 하나씩 읽음 (앞에서 뒤로)

→ 멀리 떨어진 단어는 기억이 약해짐

트랜스포머: 모든 단어를 동시에 보면서,

"어떤 단어가 어떤 단어와 관련 있는지"를 직접 연결!

예시: "오늘 아침에 비가 와서 우산을 챙겼는데... (100단어)... 결국 ___"

RNN/LSTM: 오늘 → 아침에 → 비가 → ... → 100번째 → "우산이 뭐였지?"

(순서대로 전달하다 보니 앞 내용이 약해짐)

트랜스포머: "___"에서 직접 "우산"에 주의(Attention)를 집중!
 → 중간에 100단어가 있어도 직접 연결! 안 까먹음!
 → 마치 시험 볼 때 답안지에서 교과서의 해당 페이지로 바로 점프하는 것처럼!

세 가지 비교 정리:

	RNN	LSTM	트랜스포머
기억력	짧음	중간	매우 김
긴 문장 처리	약함	보통	강함
처리 속도	느림(순차)	느림(순차)	빠름(병렬 가능)
대표 사용처	간단한 시계열	음성 인식	ChatGPT, LLM
핵심 기법	순환 연결	기억 게이트	Attention

💡 어렵게 느껴지면 이것만 기억하세요!

- **RNN** = 과거를 기억하지만, 멀리 가면 잊어버림 (전화 게임)
- **LSTM** = RNN 업그레이드. 중요한 건 오래 기억! (하지만 한계 있음)
- **트랜스포머** = 최신 기술! 모든 단어를 동시에 보면서 직접 연결! (ChatGPT가 이걸 씀!)
- 최근에는 대부분의 LLM(대형 언어 모델)이 **트랜스포머** 기반입니다!

기억하는 AI: RNN vs LSTM vs Transformer (Nano Banana Pro Edition)

단기 기억, 기억 소실, 그리고 어텐션을 통한 오래된 기억 유지



1. RNN
(Nano Banana Circuit)



2. LSTM
(Nano Banana Cell with Gate)



3. Transformer
(Nano Banana Attention Network) ✨

이번 번외편에서 배운 것 정리

김선배가 화이트보드에 정리했습니다.

번외편 핵심 정리

- ☒ 의사결정 트리 (Decision Tree) – 지도학습
 질문을 던져서 판단! "앞에 벽? → 목적지 방향은?"
 장점: 해석 가능! 왜 그렇게 판단했는지 알 수 있음
 성능: ~60-70%
- ☒ 랜덤 포레스트 (Random Forest) – 지도학습 + 앙상블
 트리 100개가 투표! 다수결로 결정!
 장점: 단일 트리보다 정확! 빠르게 학습!
 성능: ~65-75%
- 📄 RNN / LSTM / 트랜스포머 – 개념만
 RNN: 과거를 기억하지만 금방 잊는 AI
 LSTM: 중요한 기억을 오래 간직하는 AI
 트랜스포머: 모든 정보를 동시에 보는 AI (ChatGPT 등 LLM이 사용!)

[AI 도구함 – 번외편까지 배운 것들]

우리의 AI 도구함	
🔑 if문 (규칙)	- v0에서 배움, ~50%
🧠 DQN (강화학습)	- v1~v5에서 배움, 73%
♣️ 의사결정 트리	- 번외편에서 배움, ~60-70%
🌲 랜덤 포레스트	- 번외편에서 배움, ~65-75%
🧠 RNN/LSTM/트랜스포머	- 개념만! (기억하는 AI)
→ 문제에 맞는 도구를 고르는 것이 진짜 실력!	

번외편을 마치며

메타코딩은 노트를 덮으며 생각했습니다.

'DQN만 알았을 때는 망치 하나만 들고 있었는데, 이제 드라이버도, 렌치도, 톱도 생겼다.'

세상에는 DQN만 있는 것이 아닙니다. 의사결정 트리처럼 간단하지만 설명 가능한 AI도 있고, 랜덤 포레스트처럼 여럿의 힘을 모으는 AI도 있습니다.

그리고 LSTM처럼 과거를 기억하는 AI, 층을 깊이 쌓아 복잡한 판단을 하는 딥러닝도 있습니다. 아직 직접 만들어보진 못했지만, ***이런 게 있다***를 아는 것만으로도 큰 차이입니다.

문제가 생겼을 때 "DQN으로 풀어야지"가 아니라, "*"이 문제에는 어떤 AI가 적합할까?"*를 고민할 수 있게 된 것이니까요.

AI를 잘하는 사람은 알고리즘을 많이 아는 사람이 아니라, 문제에 맞는 알고리즘을 고를 줄 아는 사람이다.

— 김선배

에필로그: 실패는 다음 버전의 씨앗이다

📖 5주간의 여정을 돌아보며

금요일 저녁, 텅 빈 사무실. 메타코딩은 책상 위에 놓인 로봇 청소기를 바라보며 노트를 정리했습니다.

5주 전, 이 로봇 청소기를 처음 봤을 때의 막막함이 떠올랐습니다. 강화학습이 뭔지도 몰랐고, DQN은 이름만 들어봤고, 경사하강법은 수업 때 좋아서 기억도 안 났습니다.

그런데 막상 해보니깐요.

5주간의 여정 – 한눈에 보기

1주차 (프롤로그 + 0화: if문의 벽 + 1화: AI에게 눈을 달아주다):

프롤로그 → AI의 큰 그림, 지도/비지도/강화학습, 회귀/분류

v0 → if문으로 도전! → 복잡한 맵에서 실패

v1 → 첫 DQN! Q값, 신경망, 상태벡터 → 80% 성공!

→ 하지만 같은 맵에서만... 과적합 발견!

2주차 (2화: 넘어져야 걷는 법을 안다):

v2 → 여러 맵 학습, 랜덤 방향, 탐험 조절

→ 경사하강법, 역전파, 연쇄법칙 배움

→ 경험 리플레이로 복습 시스템 추가

→ 새 맵에서 70% 성공!

3주차 (3화: 가짜 90점의 진실):

v3 → 훈련셋/테스트셋 분리로 진짜 실력 측정

→ 90%라고 믿었던 성적이 사실 68%!

→ 과적합 방지 기법(L2, L1 규제) 배움

4주차 (4화: 처음엔 크게, 나중엔 섬세하게):

v4 → Learning Rate Scheduling으로 학습 효율 향상

→ Adam 옵티마이저 배움

→ 73% 성공! 학습 속도 25% 향상!

5주차 (5화: 현장에서 살아남기 + 번외편: 망치 너머의 도구들):

v5 → 캐싱 시스템으로 추론 속도 3.75배 향상

→ 정적/동적 환경 전략 분리

번외편 → 의사결정 트리, 랜덤 포레스트

→ RNN, LSTM, 트랜스포머 개념

→ "문제에 맞는 AI를 고르는 것이 진짜 실력!"

기술적인 것 말고, 이번 프로젝트에서 진짜 깨달은 것들이 있었습니다.

첫째: 실패가 교과서다

v0에서 if문의 한계를 발견하지 않았다면, AI의 필요성을 느끼지 못했을 것입니다. v1의 과적합 실패가 없었다면 다양한 맵 학습의 중요성을 몰랐을 것입니다. v2에서 90%라고 착각하지 않았다면 훈련셋/테스트셋 분리를 지나쳤을 것입니다. **모든 실패가 다음 버전을 만드는 재료가** 됐습니다. AI도 실패에서 배우고, 개발자도 실패에서 배웁니다.

둘째: 한 번에 완벽할 필요 없다

v0(규칙) → v1(DQN) → v2(다양한 학습) → v3(정직한 평가) → v4(효율) → v5(실전)

처음부터 v5를 만들려고 했다면?

→ 너무 복잡해서 포기했을 것

한 번에 하나씩 문제를 해결하면?

→ 규칙의 한계 → AI 도입 → 과적합 해결 → 평가 개선 → 속도 향상 → 실전 배포

→ 결국 v5까지 도달!

이것은 AI의 학습 방식과도 똑같습니다. AI도 한 번에 완벽해지지 않습니다. 수천 번의 에피소드를 거치면서 **조금씩, 조금씩** 나아지죠.

이 책에서 배운 것의 전체 분류

프롤로그에서 배운 AI의 3가지 학습 방식, 기억나시나요? 이 책에서 배운 모든 개념이 어디에 속하는지 정리해 봅시다.

이 책에서 배운 모든 개념 – 어디에 속하는 걸까?

■ 지도학습에 속하는 것:

- └─ 선형 회귀 (프롤로그)
- └─ 로지스틱 회귀 (프롤로그)
- └─ 의사결정 트리 (번외편)
- └─ 랜덤 포레스트 / 앙상블 (번외편)

■ 비지도학습에 속하는 것:

- └─ (이 책에서는 개념만! 다음 책에서 실습!)

■ 강화학습에 속하는 것 (이 책의 메인!):

- └─ Q값, DQN (1화)
- └─ 보상 함수 (프롤로그)
- └─ Epsilon-Greedy (2화)
- └─ 경험 리플레이 (2화)
- └─ 캐싱, Policy (5화)

■ 딥러닝 (도구 – 위 3가지 어디서든 사용):

- └─ 신경망 구조 (1화)
- └─ 가중치, 편향 (1화)
- └─ ReLU, Sigmoid (1화)
- └─ 경사하강법, 역전파, 연쇄법칙 (2화)
- └─ LR Scheduling (4화)
- └─ 옵티마이저 Adam (4화)
- └─ L2 규제 (3화)
- └─ RNN/LSTM (번외편 - 개념만)
- └─ 트랜스포머 (번외편 - 개념만, LLM이 사용)

◇ 모든 AI 공통 (어디서든 쓰이는 것):

- └─ 전처리, 정규화 (1화)
- └─ 과적합 / 일반화 (1화, 3화)
- └─ 훈련셋 / 테스트셋 (3화)
- └─ 평가 지표 - 정확도/성공률 (3화)
- └─ 손실 함수 (2화)
- └─ 에포크 / 이터레이션 (2화)
- └─ 학습 vs 추론 (5화)

[한눈에 보는 분류 표]

개념	지도 학습	비지도 학습	강화 학습	딥러닝 (도구)
선형 회귀	★			
로지스틱 회귀	★			
의사결정 트리	★			
랜덤 포레스트 / 앙상블	★			
DQN / Q값			★	★
보상 함수			★	
Epsilon-Greedy			★	
경험 리플레이			★	
Double/Dueling DQN			★	★
신경망 (뉴런, 가중치)				★
ReLU / Sigmoid				★
경사하강법 / 역전파				★
LR Scheduling / Adam				★
L2 규제				★
RNN / LSTM / 트랜스포머				★
전처리 / 정규화	← 모든 AI 공통! →			
과적합 / 일반화	← 모든 AI 공통! →			
훈련셋 / 테스트셋	← 모든 AI 공통! →			
평가지표 (정확도/성공률)	← 모든 AI 공통! →			
손실 함수	← 모든 AI 공통! →			
학습 vs 추론	← 모든 AI 공통! →			

★ = 이 패러다임에 속함
DQN은 강화학습 + 딥러닝이 결합된 것! (두 개에 ★)

핵심 메시지:

- "지도/비지도/강화"는 배우는 방식이 다른 것 (정답 줌 / 정답 없음 / 보상)
- "딥러닝"은 **도구(신경망)**이지, 배우는 방식이 아님
- 전처리, 과적합, 평가 등은 어떤 AI를 만들든 반드시 필요한 공통 기술
- 이 책의 DQN = 강화학습 + 딥러닝이 합쳐진 것!

📖 마지막 페이지

메타코딩의 일기 마지막 장:

처음 팀장님이 청소 로봇 AI를 만들라고 했을 때, 저는 너무 무서웠어요.

강화학습이 뭔지도 몰랐고, DQN은 이름만 들어봤고, 경사하강법은 수업 때 줄아서 기억도 안 났고...

그런데 막상 해보니깐요.

한 번에 완벽하게 할 필요가 없더라고요. v0가 실패해도 괜찮았어요. 그 실패가 v1을 만들게 해줬으니까요. v1이 실패해도 괜찮았어요. 그 실패가 v2를 만들게 해줬으니까요.

AI가 처음인 여러분에게 말하고 싶어요. 실패를 두려워하지 마세요. 실패는 다음 버전의 씨앗입니다.

모르는 개념이 나와도 괜찮아요. "나중에 이해하면 되지" 하고 계속 읽어나가다 보면 어느 순간 전체 그림이 보이기 시작해요.

한 화씩, 한 버전씩. 여러분도 할 수 있습니다.

— 메타코딩

📄 용어 사전

용어	한 줄 설명	쉬운 비유
[AI 학습 방식]		
지도학습	정답을 알려주고 배우는 AI	선생님이 답 알려주는 공부
비지도학습	정답 없이 패턴 찾는 AI	혼자서 분류하는 공부
강화학습	보상으로 배우는 AI	칭찬받으며 자라는 강아지
딥러닝	깊은 신경망 (도구!)	위 3가지 어디서든 쓰는 도구
[문제 유형]		
회귀	숫자를 예측하는 문제	"내일 기온은 몇 도?"
분류	종류를 맞추는 문제	"이건 고양이? 강아지?"

용어	한 줄 설명	쉬운 비유
선형 회귀	직선으로 숫자 예측	$y = ax + b$ (중학교 수학!)
로지스틱 회귀	확률로 분류하는 알고리즘	Sigmoid로 0~1 확률 계산
[강화학습]		
DQN	Q값 계산 신경망	AI의 뇌
Q값	행동의 예상 미래 점수	"이거 하면 얼마나 좋지?"
상태 벡터	현재 상황의 숫자 표현	AI의 눈 (11개 숫자)
보상 함수	행동의 점수를 매기는 규칙	AI의 선생님
Epsilon	탐험 확률	새 식당 vs 단골집
경험 리플레이	과거 경험 저장·복습	오답 노트
미니배치	한 번에 32개씩 학습	묶음 복습
[신경망]		
신경망	뉴런들의 연결 구조	사람의 뇌를 흉내냄
가중치	뉴런 연결의 강도	시냅스의 세기
편향	뉴런의 기본 반응값	기본 성향
ReLU	음수→0, 양수→그대로	나쁜 신호 차단 필터
Sigmoid	0~1 사이로 압축	확률 변환기
[학습 원리]		
경사하강법	오차를 줄이며 학습	안개 속 산 내려오기
Gradient	가중치 수정 방향	나침반
역전파	오차를 뒤로 전파	실패 원인 거꾸로 추적
연쇄법칙	역전파의 수학 원리	a가 b에, b가 c에 영향 → 연쇄!
손실 함수	오차 크기 측정	시험 점수 (낮을수록 좋음)
로컬 미니마	가짜 최저점	산속 작은 웅덩이
Learning Rate	학습 보폭 크기	한 걸음의 크기
LR Scheduling	보폭 자동 조절	처음엔 크게, 나중엔 작게
Adam	똑똑한 옵티마이저	프로 스키어 (지형 맞춤)
Gradient Clipping	기울기 크기 제한	급커브에서 핸들 제한
[데이터·평가]		
전처리	데이터를 숫자로 변환	외국어를 한국어로 번역
정규화	값 크기를 비슷하게 맞춤	키와 몸무게를 같은 단위로

용어	한 줄 설명	쉬운 비유
훈련셋	학습용 데이터	연습 문제집
테스트셋	평가용 데이터	시험지
정확도	성공 횟수 / 전체 횟수	우리가 쓰는 "성공률"!
과적합	암기만 하고 진짜 실력 없음	시험문제 외워서 통과
일반화	새 상황에도 잘 작동	진짜 실력
L2 규제	가중치 크기 제한	전 과목 80점 > 한 과목 100점
[학습 단위]		
에피소드	시작~종료까지 한 판	게임 한 판
에포크	전체 데이터 한 바퀴	교과서 한 번 다 읽기
이터레이션	미니배치 하나 학습	한 챕터 읽기
[추론·실무]		
학습 vs 추론	배우기 vs 써먹기	면허 학원 vs 매일 운전
캐싱	성공 경험 저장으로 빠르게	네비 즐겨찾기
Policy	AI의 행동 규칙	운전자의 판단
[번외편]		
의사결정 트리	질문으로 판단하는 AI	스무고개 게임
랜덤 포레스트	트리 100개가 투표!	반장 선거 (다수결)
앙상블	여러 모델을 모아서 판단	100명의 투표
해석 가능성	AI 판단 이유를 사람이 이해	"왜 직진?" → "벽이 없으니까!"
RNN / LSTM	과거를 기억하는 AI	중요한 기억만 오래 보관
트랜스포머	모든 정보를 동시에 보는 AI	Attention으로 직접 연결! (ChatGPT)
[고급 기법]		
Double DQN	판단+평가 분리	선수와 심판 분리
Dueling DQN	위치+행동 분리 평가	부동산: 동네 vs 이 집
PER	중요 경험 우선 복습	오답 중 자주 틀리는 것 먼저

☒ 이해도 체크리스트

AI의 큰 그림 (프롤로그)

- ☐ 지도학습, 비지도학습, 강화학습의 차이를 설명할 수 있다
- ☐ 딥러닝이 별도의 종류가 아니라 "도구"라는 것을 알겠다

- ☐ 회귀와 분류의 차이를 알겠다
- ☐ 선형 회귀($y = ax + b$)가 무엇인지 이해했다
- ☐ 로지스틱 회귀(Sigmoid)가 확률로 분류하는 것임을 알겠다

기본 개념 (0화~1화)

- ☐ if 규칙의 한계(국소 최적화)를 설명할 수 있다
- ☐ 강화학습이 왜 필요한지 알겠다
- ☐ Q값이 무엇인지 한 문장으로 말할 수 있다
- ☐ 신경망의 입력-은닉-출력 구조를 알겠다
- ☐ 전처리(Preprocessing)가 왜 필요한지 알겠다
- ☐ 정규화(Normalization)가 왜 필요한지 알겠다

활성화 함수와 학습 (1화~2화)

- ☐ ReLU가 하는 일을 알겠다
- ☐ Sigmoid가 하는 일을 알겠다
- ☐ 경사하강법을 산 비유로 설명할 수 있다
- ☐ Gradient(기울기)가 미분이라는 것을 이해했다
- ☐ 역전파가 왜 필요한지 알겠다
- ☐ 연쇄법칙(Chain Rule)이 역전파의 원리라는 것을 알겠다
- ☐ 로컬 미니마가 뭔지 이해했다
- ☐ 에피소드, 에포크, 이터레이션의 차이를 알겠다

데이터와 학습 전략 (3화)

- ☐ 훈련셋과 테스트셋을 왜 분리하는지 알겠다
- ☐ 과적합이 왜 위험한지 이해했다
- ☐ 정확도(Accuracy) = 성공률이라는 것을 알겠다
- ☐ L2 규제의 개념을 알겠다
- ☐ 경험 리플레이가 왜 필요한지 알겠다

최적화와 실무 (4화~5화)

- ☐ Learning Rate Scheduling의 원리를 안다
- ☐ Adam 옵티마이저가 SGD보다 나은 이유를 알겠다
- ☐ Learning Rate가 학습에 미치는 영향을 안다
- ☐ 학습과 추론의 차이를 알겠다
- ☐ Policy > Cache 원칙을 이해했다

번외편 (v6)

- ☐ 의사결정 트리가 "질문"으로 판단하는 AI라는 것을 알겠다
- ☐ 랜덤 포레스트가 "투표"로 정확도를 높인다는 것을 알겠다
- ☐ LSTM이 과거를 기억하는 AI라는 것을 알겠다
- ☐ 최근 LLM에는 트랜스포머를 사용한다.
- ☐ 문제에 맞는 AI를 고르는 것이 중요하다는 것을 알겠다

모두 체크했나요? 축하합니다! 이제 여러분은 AI의 기초를 이해한 person입니다!

끝.

"메타코딩과 함께한 5주간의 여정, 정말 수고하셨습니다!"